Toward World Models of Organizations: Every System Sees the Same Company

Ashwin Gopinath
Sentra, San Francisco

November 2026

Abstract

A company is one world observed by many systems. Email, chat, the CRM, support tickets, call recordings, calendars and issue trackers each record part of what is happening with every customer, deal and project, yet no system records the situation itself. We ask how a machine can learn a *world model of the organization* whose state space places every record by the situation it describes, whichever system wrote it. Our starting point is the Platonic Representation Theorem: joint-embedding predictive architectures (JEPAs) trained on *good views*, different observations of one underlying state, converge to the same representation up to a rotation. Our observation is that organizations already keep good views without anyone annotating them: records that different systems link to the same customer in the same time window. On a simulated company from Era by Eon (seven systems, 21,972 customer-linked records, exact ground truth), small text JEPAs trained from scratch on such links place 76% of records whose links were hidden with the right customer, against 53% for TF-IDF, and the same views raise a frozen pretrained sentence encoder from 53% to 62%. With the names of customers *and* employees and all identifiers masked, about half of the customer signal remains. Separately trained encoders can be partly aligned by one rotation, and a rotation lets a retrained encoder search the old index. We build access control into the model: secrets planted in records a reader may not read leak through an encoder trained on everything even when search results and states are filtered, and do not leak when each reader's model is trained only on what they may read; a shared pretrained backbone, which holds no company data, recovers part of the accuracy this costs readers who may read little. Beyond the state space, our experiments point to a division of labor among the parts of a world model. In memory, the state space should route a question to its situation and the stored records should supply the detail. When a question names its customer, a store of pretrained-encoder vectors answers as well as any combination of channels. When the question comes instead with one record of the customer and names are kept in the records, placing that record in the state space and reading its customer's records answers 63.8% of the questions, against 54.9% when a pretrained encoder places it, 52.2% with no store and 68.7% with the record's own link; with every name masked, only the record's own link helps clearly. Dynamics did not emerge from a space trained for static similarity: with monthly and weekly windows, forecasts, linear dynamics, drift and cross-team distances stay at chance or simple baselines, while similar situations across customers are found at about twice chance. We then build the decision layer on the same space, reading six kinds of logged action, conditioning the predictor on them and simulating decisions for customers in trouble. On logged data it learns little about what decisions do. This month's actions add nothing to predicting next month's state; in the link-trained space, this month's state already encodes most of them. Next month's actions help, but mostly as a description of the records next month's state is built from. And similar past situations return the situations in which the company acted rather than the effect of acting, because it almost always intervened, and in situations unlike those where it held back. We argue that learning from decisions will need actions whose assignment is known. Every encoder and head we timed trains in 2–7 minutes on a CPU, without a GPU, so at this scale a company could retrain its state space and each access scope's model daily, or whenever an access rule changes. We argue that corporate AI will run on an operational

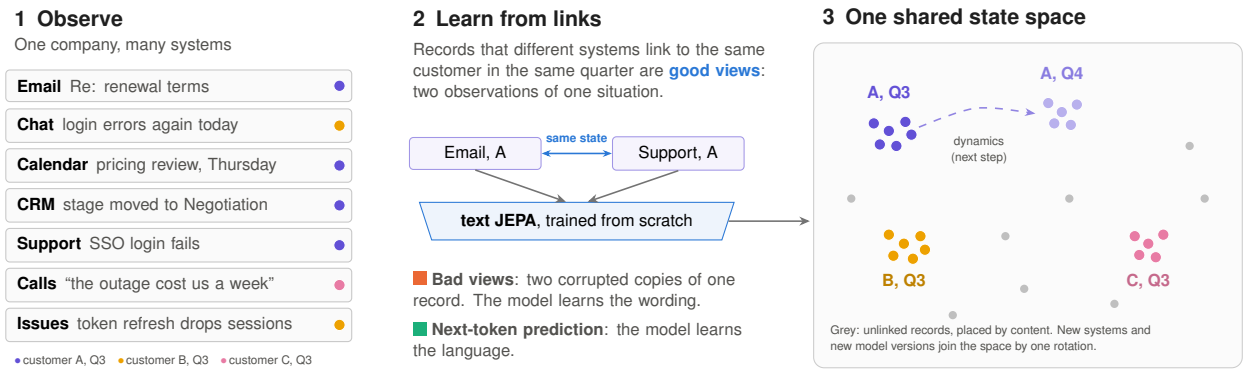


Figure 1: The idea. (1) Every system records a piece of each customer's situation. (2) The links the systems already keep tell us which records are two observations of the same situation, the *good views* that the Platonic Representation Theorem asks for. (3) A JEPA trained on them maps records from every system into one state space where records about the same customer and quarter land together and records without links can be placed by content; separately trained or retrained encoders can be partly aligned by a rotation. Predicting how states move is the next step toward a full world model.

world model of this kind, which people and agents can understand the company through, learn on and improve, and we release our code, data exporter and all results.

1 Introduction

Ask anyone in a company what is going on with its largest customer. The honest answer is spread across a sales call last Tuesday, three support tickets, a chat thread, an email chain with procurement and a CRM stage that is two weeks out of date. Each system records a piece of the situation; none records the situation itself. People rebuild it in their heads, slowly, partially and differently from one another. AI assistants rebuild it from scratch every time they are asked, by retrieving documents and reading them.

What is missing is a model of the company itself: a *world model*. In robotics and video, a world model is a learned representation of the state of the world together with a way to predict how that state changes [12, 13, 20]. For an organization, the world is the situation of every customer, deal, project and team at every moment, and the observations are the records the organization produces as it works. A world model of an organization would map every record, from every system, into one state space where records about the same situation land together. It would stay stable as systems, models and time change. Eventually, it would predict how states evolve and how actions change them.

We think every company will come to run on such a model, and that it is what corporate AI is missing: an *operational world model* that represents how the company actually functions. A capable new employee spends months learning a company from what happens around them: which customers are struggling, which deals are moving, who knows what. An operational world model lets people and AI agents learn the company the same way, from the company's own record of what happened, and lets them try a change against the model before making it in the company. Understanding the company, learning on it, and improving it all start from the same object.

The hard part is learning such a space without labels. Nobody annotates "situations". Generic text embeddings organize records by topic and template; next-token prediction learns the language of the records, not the company. The Platonic Representation Theorem [3] makes the requirement precise. JEPAs trained on **good views**, two different observations of the same underlying state, converge to the same representation up

to one rotation, whatever the modality. With *bad views*, corruptions of a single observation, they do not, and next-token prediction behaves like a JEPA with bad views. In vision and language, good views are scarce: someone has to pair an image with its caption.

A model of a company has a requirement that a model of the physical world does not: not everyone may see everything. Sales notes, support escalations, leadership discussions and private messages each have their own readers, and a model that learns from all of them can repeat what it learned to anyone who asks. Access control therefore has to be part of the model, not a filter applied to its output.

Our central observation is that **organizations produce good views for free**. The same customer appears in an email, a ticket, a call and a CRM update in the same week, and the systems themselves record the links: account identifiers, deal identifiers, organization fields, channel names, calendar invitations. Records linked to the same entity in the same time window are different observations of the same situation. The links a company already keeps are, in the language of the theorem, a supply of good views.

We test this on a simulated company generated by Era by Eon [8, 11], whose system simulators produce one consistent company with exact ground truth. We train small text JEPAs from scratch on the company's own records and compare good views, bad views and next-token prediction on identical text, architecture and budget, and we repeat the key comparisons with a frozen pretrained encoder. Our contributions are:

- A definition of an organizational world model and a recipe for its state space: states are (entity, time window) pairs, observations are records from any system, and good views are records linked to the same state (§2, §4).
- Evidence that the links make the space. On records whose links were hidden during training, good views place 76% of records with the right customer, against 53% for TF-IDF and 19% and 20% for bad views and next-token prediction (§6.2). On the training records, 95% of a record's nearest neighbours from other systems concern the same customer, against 13–14% for the alternatives (§6.1). The same good views raise a frozen pretrained sentence encoder from 40% to 72% on the training records and from 53% to 62% on hidden ones. About half of the signal survives masking every customer name, employee name and identifier (§6.3), and it degrades gracefully as links are corrupted (§6.4).
- Evidence that the space can be shared and updated. Encoders trained separately on different systems can be partly aligned by one rotation fitted on 64 anchor pairs, and encoders trained with the alternatives cannot (§6.5). A retrained encoder can search the old index through one rotation, although only the leading directions of the space survive retraining (§6.6).
- Access control built into the world model. We identify three ways a learned world model leaks across access boundaries (search results, aggregated states and the training of the encoder itself) and close all three, so that what each person receives depends only on records they may read. Planted secrets that leak through the training data even when search is filtered do not leak through the built-in design, which is non-interfering by construction. We then measure what a reader who may read little can recover without breaking the guarantee: a shared pretrained backbone helps one such reader substantially and the other hardly at all, and rotations cannot help by construction (§4, §6.7). To our knowledge, this is the first world model of an organization with access control built in.
- A test, on the same data, of what the rest of a world model needs, which points to a division of labor (§6.8–§6.10). In memory, the state space should route a question to its situation and the stored records should supply the detail: when a question comes with a record of its customer rather than the customer's name, and names are kept, placing that record with the state channel and reading its customer's records answers 63.8% of the questions, against 54.9% when a pretrained encoder does the placing; with every name masked, only the record's own link helps clearly. Dynamics did not emerge from a space trained for static similarity: forecasts, linear dynamics, drift and misalignment between teams stay at chance or

simple baselines, while search for similar situations across customers works. We then build the decision layer on the same data, reading the company's actions from its records, conditioning the predictor on them and simulating decisions for customers in trouble. Logged decisions explain little of where states go: a month's actions add nothing beyond the month's state (in the link-trained space the state already encodes most of them), and the company intervened in situations unlike those in which it held back, so similar past situations return the situations in which it acted rather than the effect of acting (§6.11). We report each result with the controls that make it interpretable.

- Code, an exporter for Era's simulators and all results (§10).

Context. At Sentra we are building operational world models of companies, on top of an organizational memory that captures how work happens across a company's systems. This paper describes building blocks of that effort, learning a shared state space and building access control into it. To keep the work reproducible and independent of any product, we study it on a public simulator.

2 What is a world model of an organization?

States, observations and dynamics. We model an organization as a set of entities e (customers, deals, projects, people) whose situation changes over time. Write $s_{e,t}$ for the latent state of entity e during time window t: everything that is true about that customer this quarter, from the open issues and the deal stage to the people involved and the mood of the last call. States are never observed directly. What the organization observes are *records* x: an email, a chat message, a ticket comment, a call-transcript turn, a CRM change, an issue, a meeting. Each record is rendered from a state by a system and carries nuisance,

$$x = r_m(s_{e,t}, n), \tag{1}$$

where m is the system (email, chat, CRM, ...) and n collects everything that is not the situation: the author, the template, the wording, the formatting. A world model of the organization has three parts:

(i) an **encoder** f that maps any record into a shared state space, $f(x) \approx g(s_{e,t})$, invariant to the system m and the nuisance n;

(ii) a **memory** that aggregates encoded records into current states and serves them to people and programs;

(iii) a **predictor** that maps a state, and possibly an action, to the next state: $s_{e,t+1} \approx h(s_{e,t}, a_t)$.

This paper is mostly about the first part, the state space, and about keeping it stable as the organization changes. We also test a first version of the second and report early, mostly negative results on the third.

What a shared state space is for. It turns everyday questions into geometry. *Where are we?* The state of a customer this week is a point, and its nearest neighbours from every system are the records that matter. The encoder can also place records that nobody linked, which in our simulated company are 72% of all records; we test this by hiding links (§6.2). *What changed?* The difference between this week's state and last week's. *What is this like?* Situations that look alike across customers, such as accounts that look the way a lost account looked before it was lost. *Is something off?* A state that moves fast, or two teams' records about the same customer drifting apart. *What about new tools and new models?* In our experiments, separately trained encoders can be partly aligned by one rotation fitted on 64 matched records, and a retrained encoder can search the old index through another (§6.5, §6.6). We test the second, third and fourth of these questions directly in §6.9 and §6.10; on this data only the third has an answer yet.

For an individual the difference is concrete. A new account manager sees one customer's situation assembled from systems they have never opened. A support engineer sees that the ticket in front of them belongs to a customer whose renewal call went badly yesterday. An executive asks what changed across all customers this week and gets an answer computed from states, not from reading thousands of documents. AI agents read a compact, shared state instead of rebuilding context from documents on every request.

Why links are the right training signal. Self-supervised learning needs a notion of "the same". Corrupting a record by cropping, masking or dropping words says that a record is the same as a damaged copy of itself; the model learns the record's wording, template and source. Predicting the next token teaches the model the language of the records. Neither says anything about the company. Links do: two records linked to the same customer in the same quarter are about the same situation, written by different people, in different systems, for different purposes, and what they share is the situation. This is the setting in which the Platonic Representation Theorem predicts convergence (§3). Links are not annotations: nobody creates them to train a model; they are metadata the systems keep in order to work. They are, however, a form of weak supervision, so we evaluate both on the records used in training and on records whose links were hidden.

3 Background

JEPAs and LeJEPA. A joint-embedding predictive architecture learns an encoder by making the embeddings of related views predictable from one another, without reconstructing the input [1, 20]. LeJEPA [2] replaces the usual heuristics against collapse with SIGReg, a regularizer that pushes the distribution of embeddings toward an isotropic Gaussian using random one-dimensional projections and the Epps–Pulley test. The loss pulls every view's projected embedding toward the mean of the global views and adds SIGReg: $\mathscr{L} = (1-\lambda)\,\mathscr{L}_{\text{inv}} + \lambda\,\mathscr{L}_{\text{SIGReg}}$.

The Platonic Representation Theorem. Huh et al. [18] conjectured that representations learned by different models and modalities converge as they scale. Balestriero and LeCun [3] turn the conjecture into a theorem for JEPAs. We use five of its parts. (1) A JEPA trained with SIGReg learns a spectral embedding of the *positive-pair graph*, the graph that says which views count as the same; the representation is set by that graph more than by the architecture. (2) Encoders trained on different modalities learn the same representation up to one rotation when their positive pairs are two observations of one scene (good views); otherwise their misalignment is bounded by how much the two graphs disagree about which scenes are alike. (3) That rotation is identified from a small number of anchor pairs by orthogonal Procrustes [31]. (4) Nuisance that the views do not share is discarded; the fraction of the embedding explained by the scene, ρ, measures how much of it is the world. (5) Next-token prediction can be read as a JEPA with bad views, parts of one document, which explains why language models align with vision only slowly as they scale. The paper validates the theory on images and captions. We test it on an organization.

4 Method

From records to states. We use customers as entities and calendar quarters as time windows, so a state is a (customer, quarter) pair; §6.9 also uses calendar months and ISO weeks. A record belongs to a state if its source system links it to the customer and its timestamp falls in the window. The links come from the systems themselves: a customer's dedicated chat channel, a customer header on an email, the CRM context of a recorded call, the organization of a support ticket, a customer field on an issue, the account of a CRM

object and, for calendar events, the call they schedule. The encoder never reads these links; they only decide which records are positives. In our experiments we keep only linked records, so that every record can be scored against the ground truth; to see how the model places records without links, we hide the links of a quarter of the threads during training (§6.2).

Views. Each batch has 256 anchor records drawn uniformly, so states are weighted by their number of records, and each anchor gets two global and six local views, following LeJEPA's multi-crop recipe. With **good views**, each view is a record drawn at random, with replacement, from the anchor's state, so the views are usually *different records*, from any system. Each is trimmed to a contiguous span (50–100% of its tokens for global views, 15–50% for local views) with 15% of tokens masked. With **bad views**, all eight views are trimmed and masked copies of the anchor record itself: standard corruption-based self-supervision for text. Everything else is identical, so the only difference is the definition of "the same".

Encoder and training. So that the encoder learns from the company's data alone, with no pretrained language knowledge, we use a deliberately small model trained from scratch: unigrams and hashed bigrams (2^{18} buckets) feed a 192-dimensional embedding bag and a two-layer MLP with a 256-dimensional output, the representation we evaluate. A projector (1,024 hidden units, 128 outputs) is used only for the loss. We train with LeJEPA's loss ($\lambda = 0.05$, 256 random slices) for 1,500 steps with batches of 256 anchor records, a peak learning rate of 10^{-3}, 5% warm-up and cosine decay, using sparse Adam for the embedding table and AdamW elsewhere. One run takes three to six minutes on a CPU. The **next-token** baseline trains the same encoder on random prefixes of records with a softmax head over the whole vocabulary (6,061 words).

A pretrained backbone with a good-view head. To test whether the recipe also improves the kind of encoder used in practice, we freeze a small pretrained sentence encoder, all-MiniLM-L6-v2 [30, 39] (384 dimensions, trained on public text and never on this company's records), and train only a head on its outputs: LayerNorm, Linear(384, 512), GELU and Linear(512, 256), with the same good views, projector and loss, for 800 steps. Because the backbone is frozen, each record is embedded once; a view is a record drawn from the anchor's state, with 10% feature dropout and Gaussian noise ($\sigma = 0.02$) in place of trimming and masking. The head takes about two minutes to train on a CPU.

Two channels: state and detail. Good views are trained to discard what the views of a state do not share, including which system wrote a record. That is what makes the space shared, and it also discards facts that are about one system's records, such as how many tickets a customer opened (§6.8). We therefore give each record two vectors: a *state* vector from good views, which says which situation the record belongs to, and a *detail* vector from a representation that is not trained to ignore the source, bad views or the pretrained encoder, which keeps what the record specifically says. We combine them in two ways. *Concatenation* normalizes each channel and puts the two side by side (Appendix B gives the normalization used in each experiment). *Routing* uses the state channel to decide which records are relevant and the detail channel to read them.

One space across systems. To test whether separately trained models agree, we split the systems into two families: messages people write to each other (email and chat) and systems of record (CRM, support, calls, calendar and issues). We train one encoder per family; each sees only its own family's records and never a cross-family pair. We whiten each encoder's output to 64 dimensions without labels and fit one orthogonal map between the two spaces by Procrustes on k anchor pairs, each a message and a record linked to the same state.

Stability over time: a rotation registry. When new data arrives, the encoder is retrained. Instead of re-embedding everything stored, we fit one rotation and one global scale from the new model's space to the old one on records both models have embedded, and rotate new queries into the old space. A registry of such rotations keeps every stored vector usable across model versions.

Access control, built in. Every record carries its *readers*, the people allowed to read it, which the source system already defines: the members of a private channel, the participants of an email thread, the teams that can open a ticket or a CRM record. Write D_u for the records reader u may read. A world model leaks across access boundaries in three ways, and each needs its own control:

(i) **Search** can return a record u may not read. Control: search only D_u, the usual fix in retrieval systems.

(ii) **Aggregation** can mix restricted records into what u sees: a customer's state vector is an average over its records, and one private escalation thread shifts it. Control: compute every state u sees from D_u only. Because states are averages, this costs one masked mean per state at read time.

(iii) **Training** can carry restricted content into the weights: an encoder trained on every record learns associations, such as a codename used only in leadership discussions and the customer it refers to, and reproduces them for anyone who queries it. Control: serve u an encoder trained only on records in D_u, with a vocabulary built from them.

With all three controls, every output u receives (search results, states, embeddings of u's own queries) is a function of D_u alone. Changing, adding or deleting any record outside D_u changes nothing u receives; this is *non-interference* [34], and here it holds by construction, not by estimate. In practice encoders are trained per access scope (a team, a role or a group of people who read the same records). The result is one world model with many views, each view complete within its reader's rights and empty outside them.

Sharing what may be shared. Per-reader encoders cost accuracy for people who may read little (§6.7). Non-interference allows a reader's model to depend, besides D_u, on anything that contains no record outside D_u. Two such things can be shared by everyone: knowledge that contains no company data at all, such as a sentence encoder pretrained on public text, and the records that *every* reader may read. We test both: a frozen pretrained backbone with a head trained per reader on D_u, and a base trained on the records everyone may read and fine-tuned per reader. Rotations between the spaces of different scopes can place different people's views in shared coordinates, since encoders trained on different record sets partly agree up to a rotation (§6.5). They respect non-interference only toward spaces the person receiving the result may read, such as the public base, or for a person entitled to both scopes: a rotation toward another scope's space is fitted with that scope's encoder, and so depends on records outside D_u. Nor can a rotation make a view more accurate. It changes the coordinates of a reader's vectors and preserves every distance between them, so a reader's search and placement are exactly as accurate after it as before; it could help only by bringing in another scope's states or index, which non-interference forbids. Rotations make views comparable; only information the reader is entitled to can make a view more accurate.

5 Experimental setup

A simulated company. Era by Eon generates synthetic companies whose systems agree with each other, behind vendor-compatible interfaces and with exact ground truth [8, 11]. We ran seven of its public simulator images (Gmail, Google Calendar, Slack, Salesforce, Zendesk, Gong and Jira) with one seed, so all seven describe the same fictional company, Northwind Robotics, as of 31 October 2026 (*day 1*). We then advanced

Table 1: The simulated company on day 1. A record is linked if its system ties it to one of the 21 active customers. Linked records define the states and the evaluation; all records can be embedded.

System	Records	Linked to a customer	Share linked
Slack (chat)	45,382	457	1%
Gmail (email)	19,713	14,160	72%
Salesforce (CRM)	5,077	1,136	22%
Zendesk (support)	4,117	4,117	100%
Gong (calls)	2,066	1,858	90%
Jira (issues)	1,206	77	6%
Google Calendar	487	167	34%
All	78,048	21,972	28%

the simulators past the close of the next fiscal quarter, 30 January 2027 (*day 2*). An exporter pulls every text-bearing record through each system's API into one table of system, timestamp, author, thread, text and link fields. Table 1 summarizes the data. On day 1 the company has 78,048 records, of which 21,972 (28%) are linked to one of 21 active customers, spread over 180 customer-quarter states from late 2022 to late 2026. By day 2 the same customers have 7,990 new linked records, written in the three months after day 1 (most in the last calendar quarter of 2026, the rest in January 2027). Era generates records from one shared company graph, with text from templates and a pre-written text pack, part of it written by a language model, whose facts are filled in from the graph [8]. Real company text is messier, a point we return to in §9.

Baselines. Bad views and next-token prediction isolate the effect of the views: same text, same encoder, same budget. TF-IDF with latent semantic analysis (256 dimensions) is a classical text representation, and a randomly initialized encoder is a floor. The frozen pretrained encoder, out of the box and with a good-view head, tests whether the recipe carries over to an encoder that already knows the language.

Metrics. *Cross-system precision@10*: for a record, the fraction of its ten nearest neighbours (cosine), restricted to records from *other systems* and other threads, that concern the same customer, or the same customer and quarter, averaged over 3,000 linked records; neighbours are searched among linked records. Restricting to other systems makes this a test of a shared state space rather than of a shared template. *Placement* (held-out records): each record is assigned to the training state whose mean embedding is nearest, and we report the share assigned to the right customer and to the right customer and quarter. *Cross-family probes*: a linear classifier trained to predict the customer from message embeddings and tested on record embeddings, and the reverse. ρ *and* R^2: on the whitened top 32 dimensions, the fraction of the embedding explained by the state, the customer or the system (between-group covariance with the within-group bias removed); ρ is the theorem's scene-explained fraction. *Alignment*: same-state recall@1 when messages retrieve records after the rotation, on held-out states, and mutual ten-nearest-neighbour agreement before it. *Time*: customer probes on new-quarter records, per-direction correlation between old and rotated new embeddings, and precision@10 of new queries against the old index. The measures of drift, misalignment and similar situations are defined where they are used (§6.10).

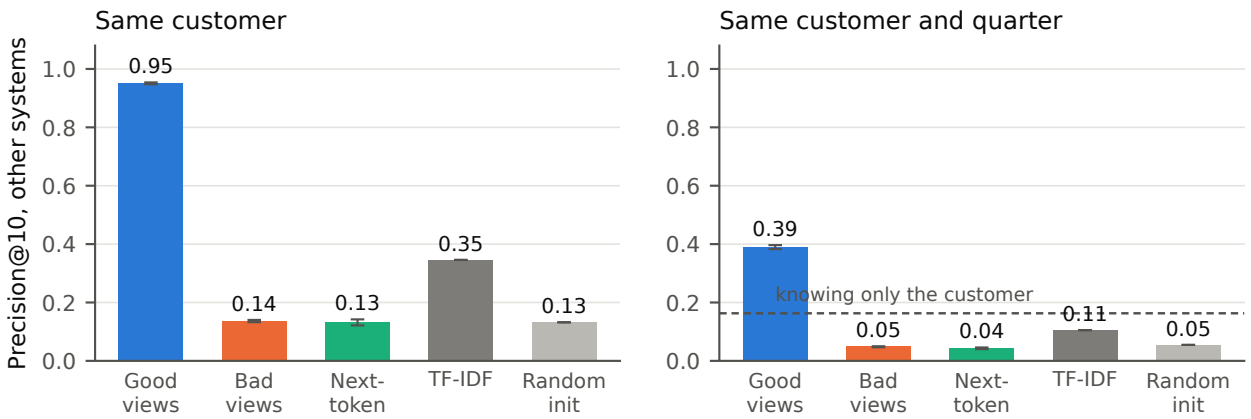


Figure 2: On the training records, only good views put records from different systems about the same situation together. Precision@10 among neighbours from *other* systems, for the same customer (left) and the same customer and quarter (right). Same text, encoder and budget for the three JEPA objectives; bars show the mean and standard deviation over three seeds. The dashed line is the most a model could reach by recognizing the customer alone.

Table 2: One encoder over all seven systems, measured on the training records (day 1). Mean ± standard deviation over three seeds for the JEPA objectives trained from scratch; one seed for the pretrained rows. Cross-system precision@10 for the same customer and the same state (customer and quarter); probe accuracy for the customer, trained on messages and tested on systems of record (M→R) and the reverse; ρ and R^2 are fractions of the whitened embedding explained by the state or by the system. The pretrained encoder is all-MiniLM-L6-v2, frozen; its head is trained on good views.

	Precision@10, other systems		Customer probe				
Representation	customer	state	M→R	R→M	ρ_{state}	R^2_{state}	R^2_{system}
Good views	**0.951** ±0.003	**0.390** ±0.007	**0.881** ±0.003	**0.927** ±0.057	**0.73** ±0.01	**0.61** ±0.01	0.04 ±0.00
Bad views	0.136 ±0.004	0.048 ±0.002	0.228 ±0.014	0.134 ±0.013	0.16 ±0.00	0.06 ±0.00	0.08 ±0.00
Next-token	0.132 ±0.010	0.043 ±0.003	0.248 ±0.012	0.210 ±0.024	0.18 ±0.00	0.06 ±0.00	0.06 ±0.00
TF-IDF + LSA	0.346	0.105	0.358	0.383	0.14	0.06	0.05
Random encoder	0.132	0.055	0.188	0.150	0.17	0.05	0.04
Pretrained, frozen	0.405	0.130	0.239	0.459	0.29	0.13	0.05
+ good-view head	0.717	0.199	0.625	0.826	0.54	0.36	0.05

6 Results

6.1 Good views absorb the company's links; bad views and next-token prediction do not

Figure 2 and Table 2 measure how each objective organizes the records it was trained on. With good views, 95.1% of a record's ten nearest neighbours from other systems concern the same customer, against 13.6% for bad views and 13.2% for next-token prediction, with the same text and model. TF-IDF reaches 34.6% and a randomly initialized encoder 13.2%: with this small encoder, corruption-based views and next-token prediction learn almost nothing about customers. For the state, customer and quarter together, good views reach 39.0% against about 4–5% for the other two objectives; a model that recognized only the customer could reach at most 16.3%, so the good-view space also separates a customer's quarters. A linear probe trained on messages names the customer of a CRM, support or call record 88% of the time with good views, and about a quarter of the time with the other objectives. The theorem's scene-explained fraction ρ is about 0.7 for good views and below 0.2 for every other representation trained from scratch, while the system that wrote a record explains about 4% of the good-view embedding. Figure 3 shows the difference: with good

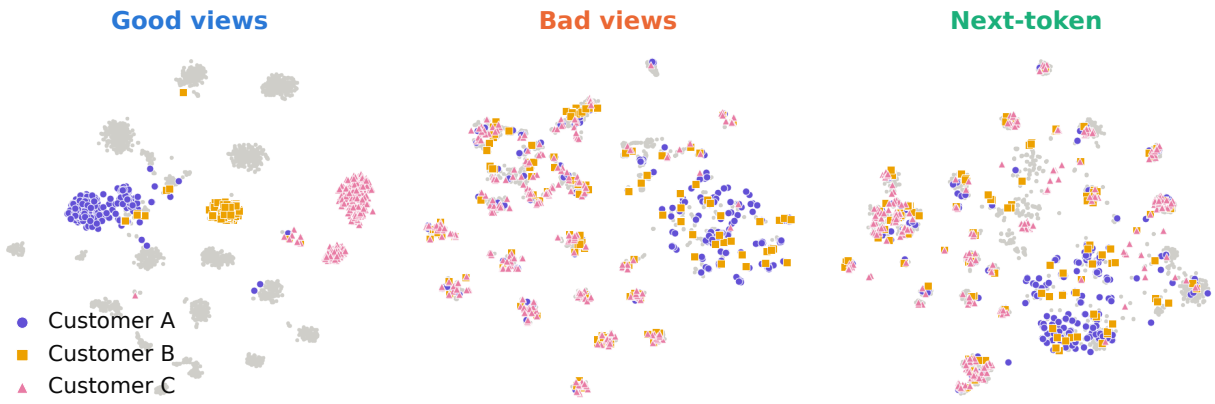


Figure 3: What each objective learns. Two-dimensional t-SNE maps of the same 3,000 training records from all seven systems. Three well-observed customers are highlighted; all other customers are grey. Only good views gather each customer's records, whatever system wrote them.

views, each customer's records from every system gather in a few tight regions; with the other objectives they are scattered among everyone else's.

The small encoder trained from scratch is a clean control, but it is not what anyone would deploy, and with it bad views and next-token prediction do no better than a random encoder. The last two rows of Table 2 repeat the measurement with a pretrained sentence encoder. Out of the box, 40% of a record's cross-system neighbours concern the same customer, more than TF-IDF. Training only a small head on its frozen outputs with good views raises this to 72%, more than doubles the cross-family probe from messages to records, and roughly doubles ρ. The recipe therefore also improves an encoder that already knows the language. On this company it does not reach the encoder trained from scratch, which can learn the company's own vocabulary (product names, error codes, people); a frozen general-purpose encoder may compress such details away.

These numbers are measured on the records used for training, with the same customer-quarter grouping that defined the positive pairs. They show how well each objective absorbs the links, not how well it generalizes. The next experiment tests generalization.

6.2 Records whose links were hidden find their place

We hid a quarter of the threads from training, 5,366 records whose links the model never saw, and trained each objective on the rest. We then placed each hidden record at the training state whose mean embedding is nearest (Table 3). Good views place 76% of the hidden records with the right customer and 26% with the right customer and quarter. TF-IDF places 53% and 14%, bad views 19% and 4%, and next-token prediction 20% and 4%. The pretrained encoder places 53% with the right customer out of the box, about as many as TF-IDF, and 62% with a good-view head trained on the same training threads. Knowing each record's customer and choosing that customer's busiest quarter would place 26% with the right state, about as many as good views do: on records the model has not seen, the gain is in finding the customer, not in telling that customer's quarters apart; monthly windows change this a little (§6.9). Among the hidden records themselves, 70% of a record's ten nearest neighbours from other systems concern the same customer with good views, against 26% for TF-IDF. Records that carry no link at all, 72% of this company's records, are in a similar position, although they may differ in kind from linked records (most are chat messages); we cannot score them directly because they have no ground truth.

Table 3: Held-out records. A quarter of the threads (5,366 records) were hidden from training: their records were never a view or a prefix. Left: precision@10 among the hidden records, neighbours from other systems. Right: each hidden record is placed at the nearest training state (mean embedding); share placed with the right customer and with the right customer and quarter. One seed.

	Precision@10, other systems		Placed at the nearest state	
Representation	customer	state	customer	state
Good views	**0.703**	**0.187**	**0.755**	**0.262**
Bad views	0.110	0.028	0.187	0.043
Next-token	0.089	0.023	0.203	0.044
TF-IDF + LSA	0.263	0.065	0.527	0.136
Random encoder	0.095	0.026	0.319	0.083
Pretrained encoder, frozen	0.257	0.071	0.526	0.128
Pretrained + good-view head	0.501	0.116	0.623	0.164
Customer known, its busiest quarter			1.000	0.264
Largest customer / largest state			0.119	0.036

6.3 More than names

A natural worry is that the encoder merely learns to spot names, as embeddings of earnings calls mostly encode company identity [41]. We masked, in every record, the customers' names and their distinctive words, contact names, email addresses, web domains and record identifiers, and retrained the good-view encoder on the masked records (one seed). On the training records, same-customer precision across systems falls from 95% to 48%, and TF-IDF falls from 35% to 14% (Figure 4, right). The company's own employees could still give a customer away, however: an account executive or support engineer who handles a fixed set of accounts is as identifying as a customer's name. We therefore also masked the employees' names, email handles and chat names, in addition to everything above, and retrained again. This mask changes 18,936 linked records, against 18,832 for the customer mask alone; most records that name an employee were already changed by it. Same-customer precision is 47% (TF-IDF: 14%), and state-level precision is 25%, still above the 16.3% that knowing only the customer would give. Employee names therefore carried almost none of the remaining signal. Roughly half of the customer signal is names and identifiers; the other half comes from what the records say.

6.4 Wrong links degrade gracefully

Real links are imperfect. Tickets get filed under the wrong organization, and duplicate customer records go unmerged. In the theorem, the damage is bounded by how much the wrong links change the positive-pair graph. We attached 25% and 50% of linked records to a random wrong state before training (one seed) and evaluated against the true states (Figure 4, left). With 25% of the links wrong, same-customer precision is 84% (from 95% with clean links) and state precision 32% (from 39%). With 50% wrong, they are 68% and 22%. Even then, same-customer precision is far above that of bad views and next-token prediction trained on clean links (14% and 13%). Wrong links blur the space; they do not break it.

6.5 Separately trained encoders can be partly aligned by one rotation

We trained one encoder on messages and another on systems of record, with no pair crossing the two, and joined them with one orthogonal map (Figure 5). After a rotation fitted on 64 anchor pairs, a message retrieves a record of the same customer and quarter as its top match 20.7% of the time on states held out from the fit,

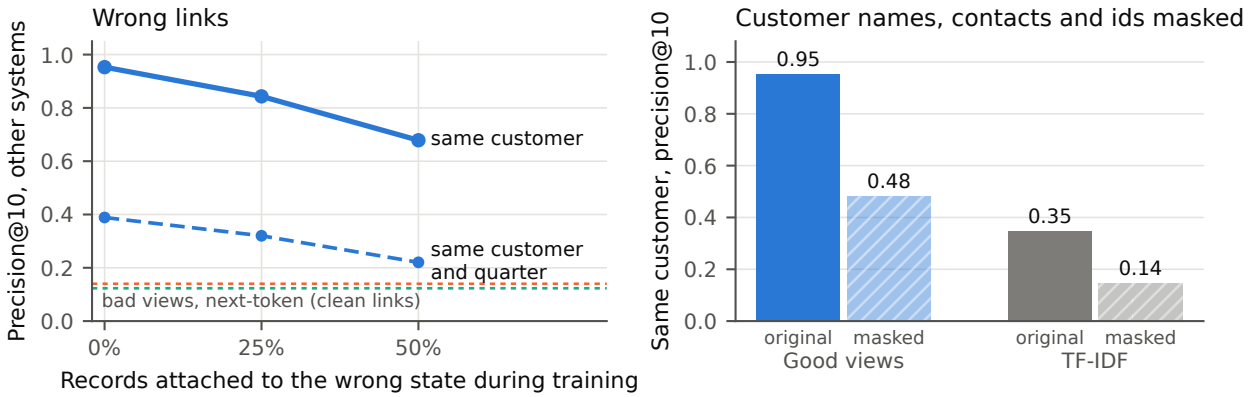


Figure 4: Robustness, on the training records. Left: good views trained with a share of records attached to a random wrong state, evaluated against the true states; dashed lines are bad views and next-token prediction with clean links. Right: same-customer precision before and after masking customer names, contacts, domains, email addresses and identifiers from every record.

against 3.8% by chance; with 2,000 anchors, 25.1%. Same-customer recall@1 is 41–50% against 14% by chance. With bad views or next-token prediction no rotation helps: same-state recall stays at 4–6%, near chance. Before any rotation, the mutual ten-nearest-neighbour agreement between the two spaces is 0.15 with good views and 0.02 with either alternative. The alignment is partial. A message finds a record of the same customer and quarter as its top match only about a fifth of the time; messages and systems of record observe different aspects of a customer's situation, while the theorem's guarantee assumes they observe the same state; and the two encoders share a vocabulary and were trained on the records of the test states, which are held out only from the rotation fit. Still, with good views a single rotation finds shared structure, and with the alternatives it finds none, consistent with the theorem's qualitative prediction.

6.6 New data and retraining

New records. A customer probe trained on day-1 embeddings assigns 72.6% of the 7,990 records written after training to the right customer with good views, against 41.0% for next-token prediction and 30.5% for bad views. The day-1 encoders never saw these records.

New models. We retrained the good-view encoder on all data through day 2, with the threads of 2,004 new records hidden from training, and fitted one rotation and one global scale from the new space to the old on 2,000 day-1 records embedded by both. Along the old space's ten leading principal directions, the old and rotated new embeddings correlate at 0.91–0.97; from the 33rd direction on, the mean correlation is 0.19 (Figure 7, left). The leading directions carry most of the variance (the first 16 carry 85%), so the rotation explains 73% of the variance, but not the low-variance tail. We then used hidden new records, which neither model was trained on, as queries against the old index. Embedded by the retrained model and rotated, they retrieve the same customer with 79.4% precision@10, against 2.9% without the rotation and 71.6% when the old model embeds them (Figure 7, right).

Not every measure favours good views. In a second retraining test, scored in a whitened 64-dimensional space that weights every direction equally, a rotation fitted on 256 anchors explains less of a retrained good-view encoder than of a retrained bad-view encoder (R^2 of -0.22 against 0.28; mutual ten-nearest-neighbour agreement 0.36 against 0.59). A plausible reading is that bad-view encoders mostly encode surface features of the text, which any retraining reproduces, while the good-view encoder's leading directions, which carry

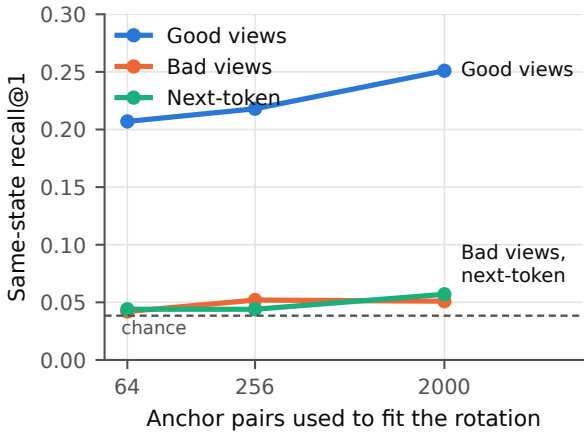


Figure 5: One rotation partly aligns two separately trained encoders, one for messages and one for systems of record, but only with good views. Same-state recall@1 of messages retrieving records, on states held out from the rotation fit.

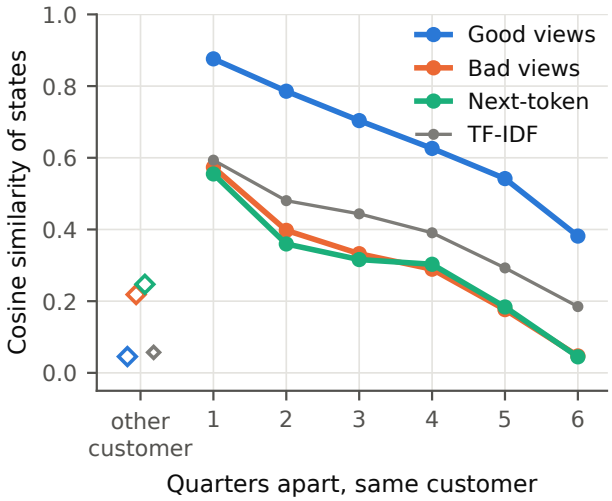


Figure 6: States persist and drift. Cosine similarity between centred state vectors of the same customer k quarters apart (lines) and of different customers in the same quarter (diamonds); states with at least five records.

Table 4: Planted secrets found by readers who may not read them. Codename search: the share of the ten records returned for a customer's codename that belong to that customer. Secret sentence: the share of the five customers most similar to the planted sentence that are the five canary customers. "Without" is the same configuration built from the corpus without the planted secrets. Encoders trained from scratch; one seed.

		Codename search, precision@10		Secret sentence, precision@5	
Reader	Configuration	with secrets	without	with secrets	without
Support engineer	no access control	1.00	0.06	0.80	0.40
	search and states filtered	1.00	0.00	0.80	0.40
	built in	0.16	0.16	0.40	0.40
Software engineer	no access control	1.00	0.06	0.80	0.40
	search and states filtered	1.00	0.00	0.60	0.20
	built in	0.04	0.04	0.20	0.20

the state, are stable and its tail is not. A rotation registry therefore preserves the part of the space that search relies on, not the whole space.

States persist and drift. We averaged each customer-quarter's records into a state vector (frozen day-1 encoders, all records through day 2) and compared states (Figure 6). With good views, a customer's states in consecutive quarters have a cosine similarity of 0.88, which falls smoothly to 0.38 six quarters apart, while different customers in the same quarter sit at 0.05. With bad views and next-token prediction, consecutive quarters are only 0.55–0.57 alike and different customers 0.22–0.25. In the good-view space a customer's state is persistent yet changes gradually. Whether those changes can be predicted is the question of §6.9.

6.7 Access control, built in

Who may read what. We gave every record its readers under a role-based policy for the simulated company (Appendix C). The CEO may read 100% of the customer-linked records, the support engineer 86%, the

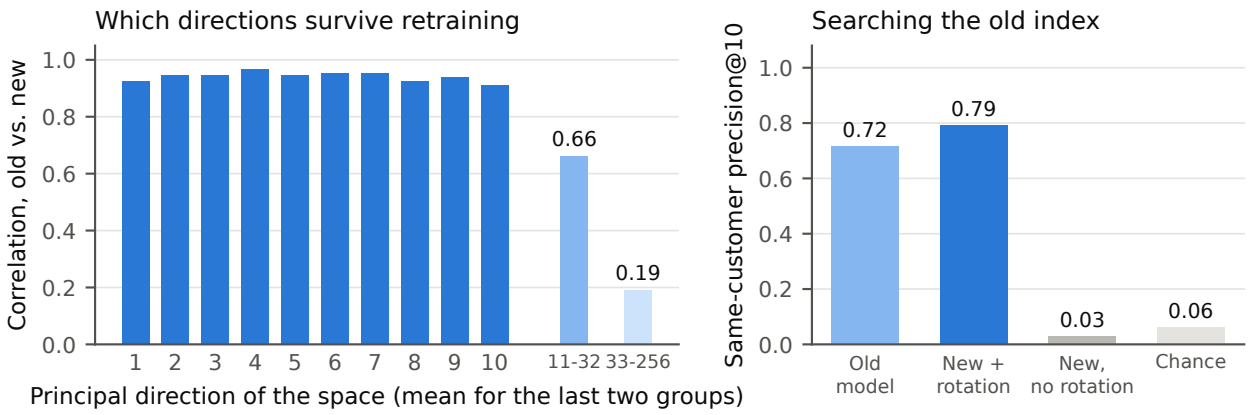


Figure 7: Retraining without re-embedding. Left: correlation between the old model's embeddings and the retrained model's rotated embeddings of the same records, along the old space's principal directions. Right: new-quarter records that neither model was trained on, searched against the old index.

account executive 81%, the software engineer 22% and the financial analyst 9%.

Leaks. We planted secrets in 150 CRM and call records of five customers, records that neither the support engineer nor the software engineer may read: a unique codename per customer, and one sentence, the same for all five, saying the customer is expected to churn to a competitor. We then measured what each of the two readers could find, and compared every configuration with the same configuration built without the secrets (Table 4, Figure 8, left); the *access advantage* is the share found with the secrets minus the share found without them. Without access control the secrets are in plain view: a codename search returns the right customer's records 100% of the time. Filtering search results and states to readable records, the usual fix, does not stop the leak. The encoder learned the codenames and the sentence from the restricted records and places them next to the right customers' *readable* records. A codename search still returns the right customer's records 100% of the time for the support engineer and 100% for the software engineer (0% and 0% without the secrets). The secret sentence puts 80% and 60% of the five canary customers among the five most similar customers (40% and 20% without). With access control built in, the readers' encoders never saw the restricted records, and the codenames are not even in their vocabularies: every result is the same with and without the secrets.

Non-interference, checked. We trained the support engineer's encoder twice, on the corpus with the planted secrets and on the corpus without them. Its training data was byte-for-byte identical, and so was every embedding it produced (largest difference: 0). The encoder trained on everything changed.

Cost. What a reader gives up depends on how much they may read (Figure 8, right). The CEO reads everything, so nothing changes. The account executive and the support engineer, who may read most records, lose little: their own encoders place held-out readable records with the right customer 76% and 74% of the time, against 78% and 75% for the encoder trained on everything. The software engineer and the financial analyst, who may read 22% and 9% of the records, lose much more (28% against 66%, and 23% against 65%): their encoders learn from few records, mostly from one or two systems.

Recovering accuracy without breaking the guarantee. The encoder trained on everything is not a fair target: its better placement of a reader's records comes partly from records the reader may not read, which is

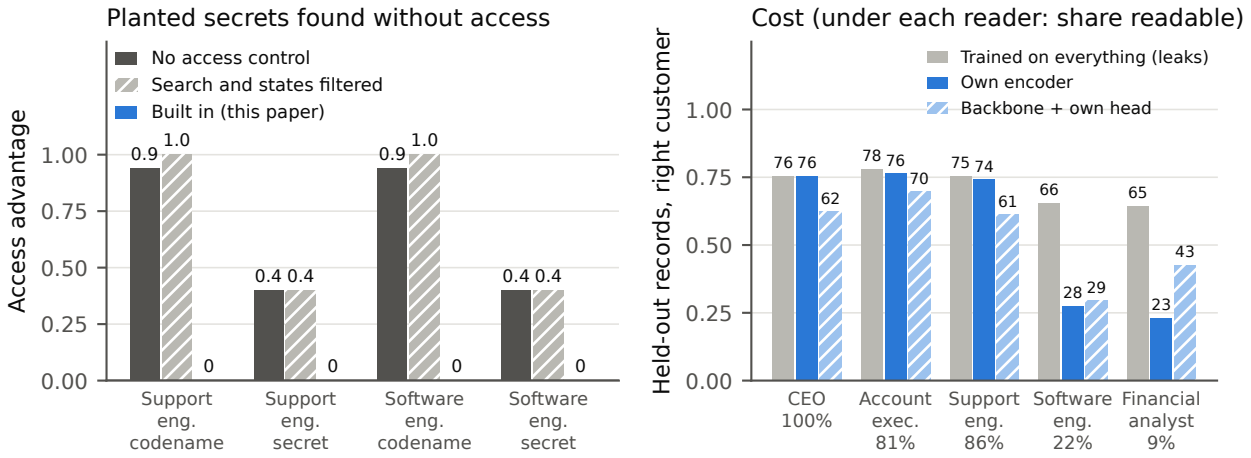


Figure 8: Access control, built in. Left: how much better two readers without access find the planted secrets (codename search and secret sentence) than they would without the secrets, with no access control, with search and states filtered to readable records, and with access control built in. Right: the share of held-out readable records placed with the right customer by each reader's own non-interfering models, an encoder trained from scratch and a pretrained backbone with a head trained on the reader's records, and by an encoder trained from scratch on everything, which leaks; under each reader, the share of records they may read.

Table 5: What a reader can recover without breaking non-interference. Share of held-out readable records placed with the right customer (nearest readable training state). "Own encoder" is trained from scratch on the reader's records; "backbone" is the frozen pretrained encoder, which holds no company data; "backbone + head" adds a head trained with good views on the reader's records; "both" puts the reader's own encoder and their backbone head side by side (run for the two readers who may read least). The last two columns are trained on every record and leak (Table 4). Best non-interfering value per reader in bold. One seed.

		Non-interfering: trained on readable records only				Trained on everything	
Reader	Readable	own encoder	backbone	backbone + head	both	encoder	backbone + head
CEO	100%	**0.76**	0.53	0.62	–	0.76	0.62
Account executive	81%	**0.76**	0.62	0.70	–	0.78	0.70
Support engineer	86%	**0.74**	0.54	0.61	–	0.75	0.61
Software engineer	22%	0.28	0.15	0.29	**0.32**	0.66	0.29
Financial analyst	9%	0.23	0.35	**0.43**	0.30	0.65	0.52

what non-interference forbids. We tested what a reader can legitimately recover with the two kinds of shared knowledge of §4 (Table 5). With a frozen pretrained backbone and a good-view head trained on each reader's records, the financial analyst's placement rises from 23% to 43%; even the backbone with no head places 35%. The software engineer barely gains: 28% with their own encoder, 29% with the backbone and a head, and 32% with both side by side. For readers who may read most records, the backbone is worse than their own encoder (62% against 76% for the CEO): a frozen general-purpose encoder with a head is weaker on this company than a small encoder trained on its records, so the backbone can help only where a reader's own data is thin, and it does not always help there. The backbone does not weaken the guarantee. With heads trained on readable records, the access advantage for codenames and for the secret sentence is +0.00 and +0.00 for the support engineer and +0.00 and +0.00 for the software engineer, and each reader's embeddings are identical with and without the secrets (largest difference 0 and 0). A head trained on every record also leaks through filtered search: the codename advantage is +0.28 and +0.40 for the two readers (+1.00 and +1.00 with the scratch encoder) and the secret-sentence advantage +0.60 and +0.60 (+0.40 and +0.40). The financial analyst may read the CRM records that carry the planted secrets, so the leak test does not apply to

Table 6: Reading facts from customer-quarter states. Accuracy of linear probes holding out one customer at a time (frozen day-1 encoders, all records through day 2). Two channels are concatenated after each is standardized and scaled to the same total variance; "system shares" adds the share of the state's records from each system. Majority class this quarter: tickets 0.57, priority 0.57, call 0.71, stage 0.29; next quarter, 0.49 on average (Appendix E). One seed.

	This quarter					Next quarter
Representation	tickets	priority	call	stage	mean	mean
Good views (state)	0.59	0.55	0.72	0.43	0.57	0.43
Bad views	0.72	0.71	0.97	0.41	0.70	0.48
Next-token	0.71	0.68	0.94	0.34	0.67	0.49
Pretrained encoder	0.65	0.67	0.91	0.29	0.63	0.48
Pretrained + good-view head	0.60	0.62	0.67	0.36	0.56	0.43
Two channels: good + pretrained	0.64	0.60	0.87	0.37	0.62	0.46
Two channels: good + bad views	0.70	0.62	0.95	0.42	0.67	0.43
Good + system shares	0.59	0.56	0.81	0.40	0.59	0.44

that reader; their guarantee, like everyone's, rests on construction.

The second kind of shared knowledge does not help here. Only 534 customer-linked records may be read by every employee (public chat channels and issues). A head trained on them alone places 17% of the software engineer's held-out records and 38% of the financial analyst's; first trained on them and then fine-tuned on each reader's records for as many steps as a head trained directly, it places 25% and 41%, against 29% and 43% for the head trained directly on the reader's records. Rotations were not run as an experiment because their effect on accuracy is fixed by geometry: placement by the nearest state centroid under cosine similarity is unchanged by any rotation of the reader's space (a unit test in the repository checks this).

Why do the two readers differ, and where does the loss come from? Almost all of what the software engineer may read (87%) is support tickets, which the backbone alone places poorly (15%, against 35% for the financial analyst's records, mostly CRM records): support tickets in this company share templates across customers, and a general-purpose encoder finds little in them that identifies the customer. To separate the number of records a reader may read from their kind, we trained the same scratch encoder on a random sample of records from all systems, as many as each reader may read (4,733 for the software engineer), and used it to place the reader's held-out records. It places 42% of the software engineer's records with the right customer (own encoder 28%, everything 66%) and 31% of the financial analyst's (23% and 65%). Both matter, and the number matters more: most of the gap to the encoder trained on everything comes from the records a reader may not read at all, and a smaller part from the fact that what they may read comes mostly from one or two systems. Neither part can be recovered from company data the reader may not read. Knowledge from outside the company can make up some of it, as the backbone does for the financial analyst; beyond that, a reader gets more only if the access policy itself gives them more, for example a summary of each customer that everyone may read. That is a decision about access, not about the model.

6.8 Two channels: state and detail

Good-view state vectors are poor at facts about which system recorded what. From a customer-quarter state they read whether a sales call took place 72% of the time, about the most common answer, while bad-view states read it 97% of the time; and a store of good-view vectors answered fewer test questions than a store of generic pretrained embeddings (Appendix F). We built the two-channel design this suggests and tested it in both places.

Table 7: Answering the 144 test questions from the store. Accuracy of the memory reader of Appendix F, closed book (no store) and reading the 16 records retrieved for each question; mean ± standard deviation over four seeds, and the gain from the store in percentage points. *The routed store retrieves with the state channel and reads the detail channel; its questions are embedded as in the detail-only store, so it shares that store's closed-book runs. The four detail-only runs happen to answer the same number of questions (102 of 144), with different answers.

Store	Seeds	Closed book	16 retrieved records	Gain
State channel only (good views)	4	0.618 ± 0.018	0.658 ± 0.028	+4.0 ± 4.1
Detail channel only (pretrained)	4	0.642 ± 0.009	0.712 ± 0.044	+6.9 ± 4.3
Two channels, concatenated	4	0.635 ± 0.009	0.722 ± 0.020	+8.7 ± 2.6
Two channels, routed*	4	0.642 ± 0.009	0.701 ± 0.023	+5.9 ± 2.2

Reading facts from states. Table 6 gives the probes of Appendix E for single channels and for two channels side by side. A detail channel restores the facts the state channel misses: whether a call took place is read 72% of the time from good views alone, 87% with the pretrained encoder as the detail channel and 95% with bad views. No combination beats the better of its two channels, however. Averaged over the four facts, good views reach 57%, the pretrained encoder 63%, bad views 70%, and good views with the pretrained encoder side by side 62%. Fact by fact, the concatenated states land between their two channels, usually close to the better one; the deal stage, which good views read best, gets worse when the pretrained channel is added and stays the same with bad views.

Answering questions from the store. We then built the memory store with four kinds of rows and trained the same reader on each, over four seeds (Table 7): state vectors only, detail vectors only (the pretrained encoder), both channels concatenated, and both channels routed, retrieving the 16 records nearest the question in the state channel and reading their detail vectors. In every store, the 16 records are those of the question's quarter whose content in the retrieval channel is most similar to the question's (Appendix F). The detail-only store answers 71.2% of the questions, the state-only store 65.8%, the concatenated store 72.2% and the routed store 70.1%. Combining the channels does not clearly beat the detail channel alone: paired over the same questions, the concatenated store is +1.0 points (95% interval −2.4 to +4.7; per seed −3.5, +4.2, +2.8, +0.7) ahead of the detail-only store, and the detail-only store +5.4 points (95% interval +0.2 to +10.2; per seed +8.3, −2.8, +4.9, +11.1) ahead of the state-only store. The lead does not come from finding the records: with the customer named in the question, the state-only and detail-only stores retrieve the asked customer's records equally often (65% and 65% of the records retrieved; 71% for the concatenated store). It comes from reading them, or from how the question is embedded; all four kinds of question (how many tickets, the worst priority, whether a call took place and the deal stage) are about one system's records, which the detail vectors describe more directly. Questions that need the state channel to find the records are absent from this benchmark; the next paragraph builds a version that has them.

When the customer must be found from a record. Every question so far names its customer, and the detail channel can find the customer's records by name. We built a version in which it cannot (Table 8; Appendix B). The question no longer names the customer; it comes instead with one record of the customer, its *anchor*, as if asked by someone holding that record: a support ticket in hand, was there a sales call with this customer this quarter? The anchor comes from a thread hidden from the state encoder's training, and from the reader's pre-training, and from a different system than the one that answers the question, so the reader must find the customer's other records across systems from one record without its link. We ran this twice, with the names of customers and employees and all identifiers masked in every stored record and question, and with them kept. A store finds 16 records of the question's quarter for the reader in one of three ways: the anchor's

Table 8: Questions that come with a record of the customer instead of its name. 127 of the 144 test questions, each anchored by one record of its customer-quarter from a held-out thread and from a different system than the one that answers it (Appendix B). Records are found among those of the question's quarter: the anchor's 16 nearest neighbours by the finding channel's content alone, the records of the customer whose centroid in the finding channel is nearest to the anchor (placement), or the records of the anchor's own customer-quarter (oracle). The reader then reads the found records' state vectors, detail vectors, or only their system and time codes. Accuracy: mean ± standard deviation over four seeds, each with its own reader and draw of anchors. Right customer: share of the 16 records found that belong to the anchor's customer. Best store without the oracle in bold, for names kept only.

		Names masked		Names kept	
Finding the records	Reading	accuracy	right customer	accuracy	right customer
No store (closed book)	–	0.520 ± 0.035	–	0.522 ± 0.010	–
Neighbours, state channel	state	0.571 ± 0.032	0.36	0.600 ± 0.024	0.67
Neighbours, detail channel	detail	0.547 ± 0.019	0.38	0.583 ± 0.019	0.53
Neighbours, both concatenated	both	0.528 ± 0.046	0.39	0.600 ± 0.017	0.63
Neighbours, state channel	detail	0.577 ± 0.010	0.36	0.600 ± 0.021	0.67
Placement, detail channel	detail	0.522 ± 0.024	0.18	0.549 ± 0.043	0.41
Placement, both concatenated	detail	0.551 ± 0.030	0.25	**0.642 ± 0.019**	0.66
Placement, state channel	detail	0.528 ± 0.036	0.25	0.638 ± 0.009	0.66
The anchor's own link (oracle)	detail	0.671 ± 0.025	0.92	0.687 ± 0.008	0.92
The anchor's own link (oracle)	state	0.699 ± 0.019	0.92	0.699 ± 0.017	0.92
The anchor's own link (oracle)	system, time	0.681 ± 0.014	0.92	0.675 ± 0.027	0.92

nearest neighbours by the content of one channel; placement, which takes the records of the customer whose centroid in that channel is nearest to the anchor (§6.2 places records at customer-quarter centroids; here the quarter comes from the question); or, as an oracle, the records of the anchor's own customer-quarter.

With names kept, finding the records with the state channel helps most. Placement with the state channel finds the right customer for 66% of the records read and answers 63.8% of the questions; placement with the pretrained encoder finds it for 41% and answers 54.9%. Paired over the same questions and anchors, the difference is +8.9 points (95% interval +3.9 to +13.8; per seed +8.7, +11.8, +11.0, +3.9). Nearest neighbours point the same way but less clearly: the state channel's neighbours are from the right customer 67% of the time and the detail channel's 53%, yet read through the detail channel they answer 60.0% against 58.3% for the detail channel's own neighbours. Neighbours are records like the anchor; placement reads a sample of the whole situation, which more often contains a record of the system that answers the question (9.8% of the records read are such records from the right customer-quarter, against 6.5% for the state channel's neighbours and 3.2% for the detail channel's). Placing with both channels concatenated does about as well as the state channel alone (64.2%). The best store without the oracle comes within 4.5 points of the oracle (68.7%) and well above no store (52.2%); paired with no store, placement with the state channel gains +11.6 points (95% interval +5.1 to +18.3; per seed +11.0, +11.8, +11.0, +12.6).

With every name masked, the customer can rarely be found from one record. The state channel still places held-out masked records with the right customer more often than the detail channel (33% against 20%). Its nearest neighbours, read through the detail channel, help a little: they answer 57.7% against 52.0% with no store, a paired gain of +5.7 points (95% interval +0.4 to +10.8; per seed +3.9, +0.8, +8.7, +9.4). Placement does not: it answers 52.8%, a paired gain of +0.8 points (95% interval −3.5 to +5.3; per seed −3.1, −2.4, −0.8, +9.4). The oracle answers 67.1%.

Which vectors the reader reads matters much less. Given the oracle's records, reading state vectors answers 69.9% of the questions with names masked and reading detail vectors 67.1% (69.9% and 68.7% with names kept). Reading only each record's system and time codes, with no content at all, answers 68.1% and

Table 9: Quarters and months. Held-out records (the threads of Table 3) placed at the nearest training state, with states defined by quarter or by month. Each good-view encoder is trained with the states it is evaluated on. One seed.

		Held-out records placed with the right	
Windows	Representation	customer	customer and window
Quarter (180 states)	good views	0.755	0.262
	customer known, its busiest quarter	1.000	0.264
Month (405 states)	good views	0.747	**0.168**
	pretrained encoder	0.527	0.071
	pretrained + good-view head	0.615	0.087
	customer known, its busiest month	1.000	0.132

Table 10: Month-to-month dynamics (469 customer-month states, 398 transitions, frozen day-1 encoders, all records through day 2). Next-month facts: mean accuracy of linear probes predicting next month's four facts from this month's state, holding out one customer at a time. Dynamics, in the 32 leading principal components of the standardized states, also holding out one customer at a time: cosine between this month's and next month's state, and the squared error of three predictors relative to predicting no change (negative is better): a linear map from the state to its change (ridge regression with $\alpha = 10$, or with α chosen by nested leave-one-customer-out validation), and shrinking the state toward the mean by one fitted factor. One seed.

	Next month	Cosine with	Squared error vs no change		
Representation	facts	next state	ridge, $\alpha = 10$	ridge, tuned α	shrink
Good views, month states	0.46	0.83	+55%	−5%	−12%
Good views, quarter states	0.44	0.90	+74%	−2%	−7%
Bad views	0.54	0.58	−27%	−28%	−28%
Pretrained encoder	0.52	0.61	−29%	−31%	−29%
Pretrained + good-view head	0.51	0.73	+2%	−19%	−19%
Two channels (good + pretrained)	0.53	0.74	−19%	−25%	−21%
Majority class	0.54				
Oracle persistence	0.72				

67.5%. For questions about how many tickets there were or whether a call took place, the provenance of the right records carries most of the answer.

What the two channels are for. The channels do different jobs. The state channel finds the situation: it places held-out records with the right customer 76% of the time against 53% for the pretrained encoder (Table 3), and in a memory, finding a question's records through it is what helps most when the question comes with a record of the customer rather than a name, at least when names are kept in the records. The detail channel finds records by what they literally say, including names, and, as §6.10 shows, finds other customers in similar trouble, which the state channel does not. Once the right records are found, their provenance carries much of what counting questions need. Keeping both channels is useful because a world model needs both abilities; blending them in one vector does about as well as the better of the two for finding and no better for answering. What a memory needs is to find a query's records through the state space, or by name when it has one, before reading them.

6.9 Finer time windows and dynamics

With quarters as windows the company has 180 states on day 1 and, with all records through day 2, 202 states and 165 quarter-to-quarter transitions, too few to learn how states change. Re-slicing the same records by calendar month gives 405 states on day 1 and, with all records through day 2, 469 states and 398 month-to-month transitions; ISO weeks give 1,445 customer-weeks with enough records to compare (§6.10).

Telling months apart. We trained a good-view encoder with customer-month states on the training threads of §6.2 and placed the hidden records (Table 9). It places 75% with the right customer and 17% with the right customer and month, against 13% for knowing the customer and choosing its busiest month. With quarters, good views did no better than knowing the customer and choosing its busiest quarter; with months, they do somewhat better, so finer windows let the model use some of the time information in the text. The pretrained encoder, with or without a good-view head, finds the right month less often than the busiest-month reference (7% and 9%).

Forecasting and linear dynamics. Month states do not predict the next month (Table 10). Probes holding out one customer at a time predict next month's facts no better than the most common answer (54% on average, against 46% for good-view month states, 54% for bad views and 53% for two channels), far below copying this month's true facts (72%). We also fitted the simplest dynamics model, a linear map from a month's state to its change into the next month (ridge regression), and compared it with predicting no change and with shrinking the state toward the average state by one fitted factor, itself a linear predictor. With a fixed, weak regularization ($\alpha = 10$) the map overfits: in the good-view space it increases the squared error by 55% relative to predicting no change. With the regularization chosen by nested leave-one-customer-out validation it reduces the error in every space (by 2–31%), but never by more than 3.4 points beyond what the single shrinkage factor achieves, and in 4 of the six spaces by no more at all. The bad-view and pretrained spaces are noisy from month to month, so the best linear guess moves the current state about halfway toward the average (shrinkage factors 0.46 and 0.51); good-view month states are smoother (factor 0.79, cosine 0.83 between consecutive months). Almost everything a linear model learns here is mean reversion: two to three times as many transitions are not enough to learn how a customer's situation moves in this data. This predictor has no actions; §6.11 adds the company's.

6.10 Operational primitives: drift, misalignment and similar situations

Section 2 promised that a shared state space turns operational questions into geometry. We tested three such questions against Era's ground truth (Table 11; details in Appendix B).

Is something off? Drift. For every customer and ISO week with at least three records, we computed a state vector and scored the week by its cosine distance from the mean of the customer's previous one to four weekly states within eight weeks (1,445 customer-weeks). If the state moves when the situation does, high scores should mark the 279 weeks with an escalation (a new urgent or high-priority ticket) or a change of deal stage, or the weeks just before them. They do not, beyond noise: the AUROC is 0.47–0.54 for every representation, within about two standard errors of chance (0.50 ± 0.04), against 0.57 for a score that uses only the change in the number of records.

Is something off? Misalignment between teams. For every customer-month with at least two sales records (CRM and calls) and two support records (tickets and issues), we computed the cosine distance between the

Table 11: Operational primitives against Era's ground truth (all records through day 2, frozen day-1 encoders). Drift: AUROC of the weekly drift score for weeks with an urgent or high-priority ticket or a deal-stage change, and for the week before one. Misalignment: AUROC of the distance between a customer's sales and support records for months with an urgent or high-priority ticket during an open deal. Similar situations: share of the five most similar states of *other* customers that share a quarter's situation (ticket count and deal stage), among all of them or only those in other quarters, and, for customers in trouble, the share of the five that are also in trouble. One seed.

	Drift, AUROC		Misalignment	Similar situations, precision@5		
Representation	same week	week ahead	AUROC	all	other quarters	in trouble
Good views, quarter states	0.49	0.49	0.48	0.18	0.17	0.04
Good views, month states	0.51	0.49	0.51	0.18	0.17	0.06
Bad views	0.53	0.54	0.53	0.21	0.17	0.35
Pretrained encoder	0.53	0.52	0.54	0.18	0.17	0.25
Pretrained + good-view head	0.49	0.48	0.57	0.13	0.12	0.10
Two channels	0.47	0.47	0.53	0.17	0.15	0.05
Record volume only	0.57	0.54	–	–	–	–
Chance or base rate	0.50	0.50	0.50	0.08	0.07	0.12

sales view and the support view of the customer (163 customer-months). If the two teams' records drift apart when a customer is at risk, the distance should be larger in the 31 months with an urgent or high-priority ticket during an open deal. It is not: the AUROC is 0.48–0.57, within two standard errors of chance (± 0.12 with so few months at risk).

What is this like? Similar situations across customers. For every customer-quarter state with at least five records (160 states), we retrieved the five most similar states of *other* customers and checked whether they share its situation, defined from the ground truth as the number of tickets (four buckets) and the deal stage (open, won, lost or none). Here every representation does better than chance: 18% of the retrieved states share the situation with good views and 21% with bad views, against 8% by chance. This is not just the calendar. Restricted to other customers' states in other quarters, precision is 17% against 7% by chance, and picking other customers in the same quarter would give 13%. For customers in trouble, quarters with an urgent or high-priority ticket during an open deal (22 states), the channels differ sharply. Among the five most similar states of other customers, 35% are also in trouble with bad views and 25% with the pretrained encoder, against a base rate of 12%, but only 4% with good views and 5% with the two channels concatenated. Trouble, defined this way, rests on facts about particular systems' records (ticket priorities and deal stages), the kind of fact the detail channel keeps and the state channel discards.

Two of the three primitives fail on this data. The third works with every representation, and for customers in trouble only with the channels that keep detail. We see three possible reasons for the failures and cannot separate them here: Era may generate escalations without precursors in the text of other systems; weekly and monthly states average few records and are noisy; and none of the encoders was trained to register change, since good views make the state stable by design and the other objectives are not trained on time at all. A world model that answers "is something off?" will probably need a predictor trained for it, on more transitions than one simulated company provides, rather than distances in a space trained for something else.

6.11 Decisions: what the company did, and what followed

Section 2 defines the predictor of a world model as action-conditioned, $s_{e,t+1} \approx h(s_{e,t}, a_t)$; the dynamics of §6.9 had no actions. The records say what the company did. For each customer and month we read six actions

Table 12: Decisions and dynamics (396 month-to-month transitions of the 21 customers in the day-2 corpus, frozen day-1 encoders, one run). Top: squared error of next month's state, in the 32 leading principal components, relative to predicting no change (negative is better), holding out one customer at a time, with the ridge strength chosen by nested validation. "State" predicts the change from this month's state alone; the next two columns add this month's six actions, as recorded and shuffled across months; the last two add next month's actions, which are recorded in next month's state itself, and next month's mix of systems, the share of its records that each system wrote. Bottom: next month's outcomes predicted from the 16 leading components of this month's state, with and without this month's actions, holding out one customer at a time; *, the 95% interval of the gain over customers excludes zero. "Actions alone" is a model of the six indicators, held out the same way; the last row scores the support-reply indicator itself.

	Squared error of next month's state, relative to predicting no change					
			+ this month's actions		+ next month's	
Representation	shrink	state	actual	shuffled	actions	system mix
Good views	−12%	−5%	−4%	−3%	−13%	−14%
Bad views	−28%	−28%	−28%	−27%	−42%	−45%
Pretrained encoder	−29%	−31%	−31%	−30%	−39%	−41%
Two channels	−21%	−25%	−24%	−24%	−35%	−38%

	Next month's outcome from this month, AUROC			
	Deal advances or is won		Urgent or high ticket	
Representation	state	+ actions	state	+ actions
Good views	0.62	0.65	0.71	0.80*
Bad views	0.63	0.65	0.78	0.80
Pretrained encoder	0.58	0.62	0.57	0.80*
Two channels	0.50	0.56*	0.52	0.78*
Actions alone		0.56		0.78
Support reply indicator		0.60		0.79

from the records and Era's ground truth (Appendix B): a sales call held (21% of customer-months), a leader, the CEO or the head of engineering, writing to or about the customer (71%), a CRM task logged (56%), an engineering escalation, a Jira issue opened for the customer (15%), support agents replying on the customer's tickets (43%), and employees writing on the customer's email threads (78%). Outcomes come from the ground truth: a deal advancing or being won (in 17% of months) and an urgent or high-priority ticket (25%). Actions and outcomes travel together: a deal advanced in 54% of the months with a sales call and 5% of the months without one. Every comparison below is observational; the company's actions were not randomized.

Action-conditioned dynamics. We added the six actions to the linear predictor of §6.9 (Table 12, top). This month's actions do not help predict next month's state. Relative to the state alone, they change the error by −0.1% to +0.6% of the state-only error across the four representations (positive is worse; good views +0.4%, 95% interval −1.6% to +2.3%); the same actions shuffled across months cost 1.0–1.4%, and this month's mix of systems changes the error by +0.2% to +1.4%. In the good-view and bad-view spaces, one reason is that the state already holds the actions: from the 16 leading components of this month's state, a logistic regression holding out one customer at a time predicts each of the six with AUROC 0.63–0.88 and 0.77–0.98. In the pretrained and two-channel spaces they are harder to read (0.49–0.86 and 0.45–0.88) and still add nothing, so redundancy is not the whole explanation. Next month's actions do help, lowering the error by a further 9–19% of the state-only error (good views 8.7%, interval 6.9–10.7%). They are, however, also a description of next month's records: a Jira record is an engineering escalation, and each of the other

Table 13: Simulating a decision for customers in trouble (29 customer-months with an urgent or high-priority ticket during an open deal). "Intervene": a sales call or an engineering escalation in the month of trouble. Each row estimates what follows the two decisions: the outcomes observed in the months in trouble; those that followed the most similar states of other customers, up to 15 in which the company intervened and 15 in which it did not among the 60 nearest (case-based simulation, in four state spaces); those of all other customers' months, without matching; and a logistic outcome model of the state and the decision. Differences in percentage points, computed before rounding, with 95% intervals in brackets from resampling customers, repeating the neighbour search and refitting the outcome model. Observational: what followed each decision, not its effect.

	Deal advances or is won next month			Urgent or high ticket next month		
Estimate	intervene	not	difference	intervene	not	difference
Observed in trouble (25 vs 4)	44%	25%	+19	68%	75%	−7
Similar states, good views ($k = 15$)	31%	19%	+12 [−14, +29]	50%	21%	+29 [+6, +45]
Similar states, bad views	27%	12%	+15 [+3, +29]	45%	37%	+8 [−6, +31]
Similar states, pretrained	26%	11%	+15 [0, +36]	45%	35%	+11 [−7, +30]
Similar states, two channels	27%	14%	+13 [−7, +28]	44%	26%	+17 [−3, +41]
All months of other customers	25%	13%	+12 [+4, +20]	36%	16%	+20 [+7, +29]
Outcome model (good views)	30%	30%	0 [−10, +6]	17%	12%	+5 [−1, +15]

actions is a kind of record of one system. Next month's mix of systems, the share of next month's records that each system wrote, lowers the error by about as much on average (10–24%), though unevenly across customers (every interval includes zero), and with the mix in the model, next month's actions lower it by a further 3–5% (the interval excludes zero only for good views). Much of what next month's actions add is what they say about the records next month's state is built from.

Outcomes. For what happens to the customer, this month's actions do add signal, but mostly through the persistence of tickets (Table 12, bottom). From this month's state, a deal advancing next month is predicted with AUROC 0.50–0.63 across the four representations and an urgent ticket with 0.52–0.78. Adding this month's actions changes these by +0.02 to +0.07 and +0.02 to +0.26; the interval excludes zero in one of the four representations for deals and in three for urgent tickets, and the gain is largest where the state is weakest. The signal sits in support replies: adding only them to the state gives the same AUROC as adding all six actions (differences −0.02 to +0.02, every interval including zero), and the reply indicator by itself ranks next month's urgent tickets with AUROC 0.79. A support reply marks a month with a ticket (99% of months with a ticket have one, 6% of months without), and the indicator of any ticket this month ranks next month's urgent tickets as well (AUROC 0.79). Customers with tickets keep having them, and the reply mostly marks that persistence rather than the effect of a decision.

Around single actions. At a finer scale, we compared the good-view state in the 14 days before and after an action with the same customer's state around a time with no action of that kind within 14 days, leaving all records of the action's kind out of both windows. The actions are the 216 sales calls (at 20 customers), 101 engineering escalations (11 customers) and 66 first leader messages after a quiet fortnight (19 customers) that have at least two other records of the customer on each side. The state moves more around engineering escalations than at matched times: $1 -$ cosine is 0.037 against 0.026, a difference of +0.011 (interval +0.004 to +0.020). Their windows hold more records, though (a median of 64 before the event against 46), and with each window's record count accounted for, the difference is +0.009 (interval −0.003 to +0.017). Around sales calls it is 0.000 (interval −0.008 to +0.010), and around leaders' messages −0.024 (interval −0.065 to +0.022), from only 29 matched times, because leaders write about most customers often. Knowing which action was taken does not help predict where the state goes either: across 729 pairs of windows, it changes

the error of a linear predictor by +0.02%.

Simulating a decision. For the 29 customer-months in trouble, an urgent or high-priority ticket during an open deal, we asked the question a world model of a company is for: what follows if the company intervenes, with a sales call or an engineering escalation, and what if it does not (Table 13)? The months come from eight customers, 12 of them from one. The machinery works: the state space retrieves similar situations of other customers, splits them by what the company did, and returns what followed. The answers cannot be taken as effects, and the data show why. First, the logs barely contain the alternative: the company intervened in 25 of the 29 months and held back in 4. Second, decisions follow situations that the state does not fully capture. Whether the company intervened in a month can be predicted from the month's state (AUROC 0.68–0.94 across the four representations, holding out one customer at a time), yet among the most similar states of other customers, those in which the company intervened were themselves in trouble 19–24% of the time and had an urgent ticket 57–62% of the time across the four spaces, against 0.2–4% and 17–35% for those in which it did not (in the good-view space, 24% and 60% against 0.2% and 17%). The case-based comparison therefore sets troubled months against calm ones, and intervening appears to be followed by more urgent tickets (+29 points, interval +6 to +45), which is what one would see if calls and escalations are made when problems are severe, whatever their effect. The estimates also move with the space used to find similar states (+12 to +15 points for deals and +8 to +29 for urgent tickets, with six of the eight intervals including zero), and the outcome model, which takes the decision as an input, attributes little to it (0 and +5 points). Within the months in trouble, the first call or escalation came before the month's first urgent ticket in 13 and after it in 12, so a monthly action indicator mixes responses to a ticket with actions taken before it.

The decision layer of an operational world model can thus be built on the state space from the company's own records, without new data: actions, action-conditioned transitions, event windows and decision simulation. On one company's logs it learns little about what decisions do. This month's actions add nothing to next month's state, and in the link-trained space the state already encodes most of them; next month's actions help mostly by describing the records next month's state is built from; no single action moves the state detectably more than matched times once record counts are accounted for; and the company intervened in situations unlike those in which it held back, in ways that matching on the state did not remove in any of the four spaces. Learning from decisions will need actions whose assignment is known: decisions logged with their reasons, deliberately varied, or replayed in a simulator that can run the same situation with a different decision.

7 Why this matters: from records to a model of the company

Most enterprise AI today is retrieval: given a question, find documents, read them, answer and forget. Every question pays the full cost of rebuilding the situation, and two questions about the same customer may rebuild it differently. A world model inverts this. The organization's state is maintained continuously as records arrive, and questions are answered from the state. The path from records to such a model has four steps, each useful on its own and each making the next possible: (1) a shared state space learned from the links an organization already keeps, the subject of this paper; (2) a memory that stores both the state and the detail of every record and serves each question from the channel suited to it; (3) a predictor of how states change, trained on the organization's own history of states, decisions and outcomes; and (4) planning, in which people and agents try decisions against the predictor before making them. Four properties make the first step practical, and the experiments above speak to each, with the caveats noted.

- **No annotation.** The training signal is the link structure the organization's systems already keep (§6.1, §6.2). Nobody labels anything, wrong links blur the space instead of breaking it (§6.4), and the same signal improves an off-the-shelf pretrained encoder (§6.1).

- **Records without links find their place.** Records whose links were hidden during training are placed with the right customer by what they say (§6.2), and masking the names of customers and employees in the training records shows that the space is built from more than names (§6.3).
- **Every system, one space.** Encoders trained separately for different systems can be partly aligned by one rotation fitted on matched records (§6.5). A new tool, or an acquired company's chat, could join this way without retraining everything; we have not tested that directly.
- **Memory that survives upgrades.** One rotation lets a retrained encoder search the old index (§6.6). If that holds across many upgrades, which we have not tested, an organization's memory can compound across model versions instead of being rebuilt with each one.

A division of labor. The experiments draw sharper lines between the parts of a world model than we expected. The state space, the first part, can be learned from links alone, places records that carry no link, can be shared across encoders and versions through rotations, and can carry access control. Memory, the second part, should find through the state space and read from the store. When a question names its customer, the records can be found by name; when it comes with a record instead, finding the customer's records through the state space is what helps, clearly when the record is placed in the space and less clearly through its neighbours, and the store's job is then to supply what those records say and where they came from (§6.8). The third part, a predictor, did not emerge here from a space trained for static similarity: with 2.4 times as many transitions, linear models learn little beyond mean reversion, and drift and misalignment scores are at chance (§6.9, §6.10). Conditioning it on the company's logged actions does not change that, and simulating decisions from the logs returns the situations in which the company acted rather than the effect of acting (§6.11). We expect a predictor to need transitions from many companies and actions whose assignment is known: logged with their reasons, deliberately varied, or replayed in a simulator. These lines are drawn on one simulated company and may move with more data.

Cheap enough to retrain often. Every encoder and head we timed trains in 2–7 minutes on a CPU, without a GPU: the state encoder from scratch, each reader's encoder, the head on the pretrained backbone and the month encoder (each reader's head on the backbone, which we did not time, is the same head trained on fewer records). The memory reader, pre-training included, took at most 16 minutes. At this scale a company could retrain its state space, and the model of every access scope, daily or whenever an access rule changes, where adapting a large language model to the same data would typically need GPUs. That matters for corporate AI, where the data change daily and every change must reach only the people allowed to see it. Whether training stays this cheap for a company with millions of records is untested.

Access control is part of the model. Enterprise systems took decades to encode who may see what. A world model that flattens those rules into one shared representation undoes that work, however carefully its search results are filtered: our experiments show restricted content leaking through the encoder's weights even when every search result and every state shown is computed from readable records (§6.7). Aggregation is a third path that a filtered index does not close; we argue it from the construction of states rather than measure it separately. Building the three controls into the model instead keeps each person's view complete within their rights and empty outside them, with a guarantee that holds by construction. It also makes the model easier to trust and to audit: deleting a record, and retraining the encoders of the scopes that could read it, removes its influence from every view that could see it, and from no other. The guarantee has a price for people who may read little, and knowledge without company data repays it only partly. What a support ticket means is learned largely from the emails, calls and CRM records around it, and a reader who may not read those cannot have that knowledge from the model either (§6.7).

What changes for a company. When people and AI agents read the same state, hand-offs, onboarding and reviews start from the state instead of from a search. Because the state space is shared, what the organization knows accumulates in one place instead of in each person's head, and it stays usable when people leave. Whether coordination becomes measurable, for instance whether two teams whose records about the same customer drift apart are misaligned before anyone notices, is a hypothesis we could not confirm on this data (§6.10).

Toward dynamics. A state space is the first half of a world model. The second half is a predictor over it: given a customer's state this quarter and an action, such as a discount, an escalation or a new owner, what is the likely next state? Predictors trained in a learned latent space are how world models are built for video and robotics [13, 20]. In an organization, the training data for such a predictor is the history of states, decisions and outcomes that the company already records. Our results show that 165 quarterly or 398 monthly transitions from one company are not enough: a linear predictor learns little beyond mean reversion; many companies, explicit actions, and objectives that reward predicting change rather than holding the state still will be needed. Our attempt to condition the predictor on the company's own actions (§6.11) suggests what such training data must contain: decisions whose assignment is known, because they were logged with their reasons, varied on purpose, or replayed in a simulator. With a predictor, a company can ask "what happens if" before acting, and agents can plan against a model of the organization instead of against documents.

Personal world models. The same recipe applies to one person. Their email, calendar, messages and documents are different observations of their own projects and relationships, linked by people, threads and time.

Why we expect every company to have an operational world model. Every company already has a world model: it is distributed across the heads of its people, partial, inconsistent and lost when they leave. Making it explicit changes three things. People and agents can *understand* how the company is functioning from one shared, permission-respecting state instead of reassembling it from documents. They can *learn* on it the way a person learns a company, from its own history of situations, decisions and outcomes, with a predictor over the state space as the learning substrate. And they can *improve* the company by trying a decision against the model before making it, and by seeing where the company's actual trajectory departs from the one it planned. The state space and the access control studied here are the foundation those three uses share; the predictor that the second and third uses need is the open problem our negative results point to.

8 Related work

Linked records as positives. Treating records that share a link as positive pairs is established: citations [7], authors and venues [42], product identifiers [27], foreign keys in relational databases [28], one bank client's transactions, locations and support calls [23], one patient's codes and notes [5] and, in industry, per-company embedding models fine-tuned on document links, co-access and chat threads.[1] The closest is the multimodal banking work, which trains one encoder per modality jointly with a CLIP-style objective [29]. We differ in defining states by entity *and* time window across a company's systems, training per-system encoders separately and joining them by a rotation, and testing the predictions of the Platonic Representation Theorem against bad-view and next-token controls. Where the industrial practice fine-tunes a pretrained sentence

[1] https://jxnl.co/writing/2025/03/06/fine-tuning-embedding-models-for-enterprise-rag-lessons-from-glean/

encoder [30], we also train only a head on a frozen one, which keeps the backbone free of company data and therefore shareable across access scopes.

JEPAs for text. LLM-JEPA applies a JEPA loss to curated pairs of text and code that describe the same thing [16], and Zuo et al. [43] apply JEPA with SIGReg to sections of a scientific paper. Our views come from organizational links, and our encoder is trained from scratch on one company.

Convergence and alignment of representations. Beyond the Platonic hypothesis and theorem [3, 18], anchors and orthogonal maps join independently trained latent spaces [21, 24] and data silos [26], and orthogonal Procrustes has aligned word embeddings across decades [14]. Theory relates self-supervised representations to the spectrum of the positive-pair graph [15] and to the content that views share [37].

Compatible embeddings across model updates. Backward-compatible training makes a new model's embeddings comparable with an old index [33]; fitted Procrustes maps or affine adapters do so after training [22, 36]. Our rotation registry is an instance of the latter; for good-view encoders it preserves the leading directions of the space but not the tail.

Access control in machine learning. Information flow control for machine learning trains one expert module per security domain and combines only the experts a user may access, which gives non-interference for language models [34]. Permissioned LLMs attach parameter-efficient adapters to security domains and measure leakage with an access-advantage metric [19], and a position paper argues that enterprise models must be trained, and retrieve, only on content that every participant in an interaction may access [4]. Permission-aware retrieval, which filters search results by the reader's rights, is standard industry practice. We bring these ideas to a learned world model of an organization, where a third leak path appears, aggregation of records into states, and we measure what readers who may read little can recover from shared knowledge without breaking non-interference.

Memory for language models. Memorilla compresses a document collection into a few query-dependent memory tokens for a frozen model [9], and xRAG compresses a retrieved document into one token [6].

Simulated organizations and enterprise world models. Era by Eon [11], OrgForge [10], TheAgentCompany [40] and CRMArena [17] simulate companies to evaluate agents; we use one to learn representations. Nair et al. [25] study learned world models of enterprise systems as predictors of state transitions and find that they falter when a deployment's configuration changes. Organizational memory has a long history in management research [38], and object-centric process mining relates events to the business objects they touch [35]. For individuals, general user models are built from a person's computer use and kept as text statements [32].

9 Limitations

- **One simulated company.** All results come from one Era company with 21 active customers. Era's text is cleaner and more regular than real company text, and its systems agree by construction. Replication on more simulated companies, and on real data, is the most important next step. Several negative results (drift, misalignment, forecasting) may reflect how Era generates events rather than what a state space

can do: if escalations have no precursors in the text of other systems, no representation of that text can anticipate them.

- **Weak supervision, evaluated partly on itself.** Links are not annotations, but they are supervision: the positive pairs come from the same customer-quarter grouping that most of our metrics score. Results on training records (§6.1, §6.3, §6.4) are therefore upper bounds; the held-out test (§6.2) is the fairer measure.
- **Weak alternatives from scratch.** With our small encoder, corruption-based views and next-token prediction learn little about customers on training records (no better than a random encoder there). The pretrained encoder is a stronger baseline, and good views improve it too, but we tested only one small, frozen backbone; fine-tuning a larger encoder end to end may behave differently.
- **Seeds.** The main comparison uses three seeds and the memory reader four; the held-out, alignment, time, masking, noise, pretrained, access-control, primitive and decision experiments use one.
- **Partial alignment.** Separately trained encoders agree on the leading directions, not on the tail, and messages and systems of record share only part of the state.
- **A narrow, partly constructed question benchmark.** The 144 test questions ask only about counts and categories within one system's records. The anchored version is constructed to need the state channel: its anchors come from held-out threads and from systems other than the one that answers the question, retrieval takes the quarter from the question, and questions without such an anchor are dropped (127 of 144 test questions remain). Its four seeds vary the reader and the anchors but not the state encoder, which is one run. Questions that need several systems' records at once, or that come with several records of the customer, remain untested.
- **No dynamics yet.** Neither forecasts nor a linear predictor beat simple baselines with quarterly or monthly windows (§6.9), conditioning the predictor on logged actions does not change that (§6.11), and drift and misalignment scores show no signal beyond noise (§6.10).
- **Actions read from logs with simple rules, and observed rather than assigned.** The six actions are read with simple rules (a support agent is anyone who comments on the tickets of at least three customers; leaders are the CEO and the head of engineering), and the company's decisions were not randomized, so the decision simulation estimates what followed each decision, not its effect; the 29 months in trouble come from eight customers (§6.11).
- **Access-control policy.** Our policy is role-based and built from what our export kept (system, channel privacy, direct-message membership, sender, organizer). Real deployments have finer rules (channel membership, email recipients, internal notes, record sharing), which plug into the same mechanism but were not tested. Per-scope models also cost one training run per scope, and very small scopes may not have enough records to train one; a shared backbone helps some of them (§6.7).

10 Conclusion

Organizations already keep the training signal for a model of themselves. Records that different systems link to the same customer in the same window are good views of one situation. Training on them, with no annotation, yields a state space that places records from every system by the situation they describe, generalizes to records whose links it never saw, keeps about half of its customer signal when the names of customers and employees are masked, improves an off-the-shelf pretrained encoder, can be partly aligned across separately trained encoders, and lets a retrained encoder use an old index. Access control can be built

into the same model, so that each person's view depends only on what they may read; knowledge without company data can be shared to soften the cost for people who may read little, and rotations can make views comparable but not more accurate. What the state space does not do is just as clear on this data: read through a linear probe, it is poor at facts about one system's records, which a memory recovers from the records it finds, so a memory should find through the state space and read from the store; and nothing we built on it predicts how a customer's situation will move beyond mean reversion and the persistence of tickets, even with the company's own actions in hand: the company intervened in situations unlike those in which it held back, so its logs show the situations in which it acted rather than the effect of acting. We expect every company to come to run on an operational world model of this kind, one that people and agents can understand the company through, learn on and improve. At Sentra we are building it, and this paper lays down its first layers, the state space and the access control built into it, and marks where the next ones must start: a memory that routes each question to its situation before reading, and a predictor of how states change and how actions change them, trained on transitions rather than on similarity and on decisions whose assignment is known.

Code and data. Code, the Era exporter, every result reported here and the scripts that draw the figures and fill in every number of this paper are released with it.[2] Era's simulators are free; the exact company can be regenerated from the seed and dates in the repository.

Acknowledgments. We thank Eon for making Era freely available. Experiments and drafting were assisted by an AI model (Claude, Anthropic); the author takes full responsibility for the content.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[2] Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.

[3] Randall Balestriero and Yann LeCun. Proving and improving the Platonic representation hypothesis with JEPAs. *arXiv preprint arXiv:2610.39302*, 2026.

[4] Shashank Shreedhar Bhatt, Tanmay Rajore, Khushboo Aggarwal, Ganesh Ananthanarayanan, Ranveer Chandra, Nishanth Chandran, Suyash Choudhury, Divya Gupta, Emre Kiciman, Sumit Kumar Pandey, Srinath Setty, Rahul Sharma, and Teijia Zhao. Position: Enterprise AI must enforce participant-aware access control. *arXiv preprint arXiv:2509.14608*, 2025.

[5] Tianxi Cai et al. Contrastive learning on multimodal analysis of electronic health records. *arXiv preprint arXiv:2403.14926*, 2024.

[6] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[7] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. SPECTER: Document-level representation learning using citation-informed transformers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[2] Repository: `company-world-model` (link to be added on release).

[8] Eon. Meet Era: Simulated companies to benchmark models and eval agents. https://www.eon.io/blog/era-build-and-test-ai-agents, 2026.

[9] Adibvafa Fallahpour, Parsa Idehpour, Nikita Mounier, Shirley Wu, Shayan Pardis, Vignesh Kothapalli, and Jure Leskovec. Memorilla: Latent semantic memory for LLMs. *arXiv preprint arXiv:2610.21375*, 2026.

[10] Jeffrey Flynt. OrgForge: A multi-agent simulation framework for verifiable synthetic organizational corpora. *arXiv preprint arXiv:2603.14997*, 2026.

[11] Benjamin Gruenbaum, Doron Porat, Assaf Natanzon, Roy Zavida, Chen Dinachi, and Or Itzahary. The Era by Eon benchmark: A generated enterprise estate with exact ground truth for benchmarking LLM agents. *arXiv preprint arXiv:2609.09853*, 2026.

[12] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[13] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

[14] William L. Hamilton, Jure Leskovec, and Dan Jurafsky. Diachronic word embeddings reveal statistical laws of semantic change. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.

[15] Jeff Z. HaoChen, Colin Wei, Adrien Gaidon, and Tengyu Ma. Provable guarantees for self-supervised deep learning with spectral contrastive loss. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[16] Hai Huang, Yann LeCun, and Randall Balestriero. LLM-JEPA: Large language models meet joint embedding predictive architectures. *arXiv preprint arXiv:2509.14252*, 2025.

[17] Kung-Hsiang Huang et al. CRMArena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments. *arXiv preprint arXiv:2411.02305*, 2024.

[18] Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The Platonic representation hypothesis. In *International Conference on Machine Learning (ICML)*, 2024.

[19] Bargav Jayaraman, Virendra J. Marathe, Hamid Mozaffari, William Shen, and Krishnaram Kenthapadi. Permissioned LLMs: Enforcing access control in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[20] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[21] Valentino Maiorca, Luca Moschella, Antonio Norelli, Marco Fumero, Francesco Locatello, and Emanuele Rodolà. Latent space translation via semantic alignment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[22] Lucas Maystre et al. When embedding models meet: Procrustes bounds and applications. *arXiv preprint arXiv:2510.13406*, 2025.

[23] Dzhambulat Mollaev, Ivan Kireev, Mikhail Orlov, Alexander Kostin, Ivan Karpukhin, Maria Postnova, Gleb Gusev, and Andrey Savchenko. Multimodal banking dataset: Understanding client needs through event sequences. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025.

[24] Luca Moschella, Valentino Maiorca, Marco Fumero, Antonio Norelli, Francesco Locatello, and Emanuele Rodolà. Relative representations enable zero-shot latent space communication. In *International Conference on Learning Representations (ICLR)*, 2023.

[25] Jishnu Sethumadhavan Nair, Patrice Bechard, Rishabh Maheshwary, et al. Do enterprise systems need learned world models? The importance of context to infer dynamics. *arXiv preprint arXiv:2605.12178*, 2026.

[26] Keiyu Nosaka et al. Data collaboration analysis with orthonormal basis selection and alignment. *arXiv preprint arXiv:2403.02780*, 2024.

[27] Ralph Peeters and Christian Bizer. Supervised contrastive learning for product matching. *arXiv preprint arXiv:2202.02098*, 2022.

[28] Jakub Peleška and Gustav Šír. Task-agnostic contrastive pretraining for relational deep learning. *arXiv preprint arXiv:2506.22530*, 2025.

[29] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[30] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[31] Peter H. Schönemann. A generalized solution of the orthogonal Procrustes problem. *Psychometrika*, 31 (1):1–10, 1966.

[32] Omar Shaikh, Shardul Sapkota, Shan Rizvi, Eric Horvitz, Joon Sung Park, Diyi Yang, and Michael S. Bernstein. Creating general user models from computer use. *arXiv preprint arXiv:2505.10831*, 2025.

[33] Yantao Shen, Yuanjun Xiong, Wei Xia, and Stefano Soatto. Towards backward-compatible representation learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[34] Trishita Tiwari, Suchin Gururangan, Chuan Guo, Weizhe Hua, Sanjay Kariyappa, Udit Gupta, Wenjie Xiong, Kiwan Maeng, Hsien-Hsin S. Lee, and G. Edward Suh. Information flow control in machine learning through modular model architecture. In *USENIX Security Symposium*, 2024.

[35] Wil M. P. van der Aalst. Object-centric process mining: Dealing with divergence and convergence in event data. In *Software Engineering and Formal Methods (SEFM)*, 2019.

[36] Harshil Vejendla. Drift-adapter: A practical approach to near zero-downtime embedding model upgrades in vector databases. *arXiv preprint arXiv:2509.23471*, 2025.

[37] Julius von Kügelgen, Yash Sharma, Luigi Gresele, Wieland Brendel, Bernhard Schölkopf, Michel Besserve, and Francesco Locatello. Self-supervised learning with data augmentations provably isolates content from style. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[38] James P. Walsh and Gerardo Rivera Ungson. Organizational memory. *Academy of Management Review*, 16(1):57–91, 1991.

[39] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, pages 5776–5788, 2020.

[40] Frank F. Xu et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

[41] Ding Yu, Zhuo Liu, and Hangfeng He. Same company, same signal: The role of identity in earnings call transcripts. *arXiv preprint arXiv:2412.18029*, 2024.

[42] Yu Zhang et al. Metadata-induced contrastive learning for zero-shot multi-label text classification. *arXiv preprint arXiv:2202.05932*, 2022.

[43] You Zuo, Éric de la Clergerie, and Benoît Sagot. Asymmetric within-document predictive learning for scientific document representation. *arXiv preprint arXiv:2608.28625*, 2026.

A Experimental details

Data export. The seven simulators run as Docker images (`erabyeon/*`) with seed 42, the simulated date set to 31 October 2026 and high fidelity. The exporter reads every system through its REST interface: Slack messages, Gmail messages, Google Calendar events, Salesforce objects (accounts, opportunities, tasks, cases and notes), Zendesk tickets and comments, Gong call transcripts and Jira issues and comments. Day 2 is the simulators' second state, after the fiscal quarter that closes on 30 January 2027. Texts are lower-cased and tokenized into words; each record keeps its first 96 tokens, and the vocabulary keeps words that occur at least twice among the linked records (6,061 words on day 1; it is built before any held-out split, from text only). All experiments use linked records only.

Hyperparameters. Encoder: embedding bag of 2^{18} buckets $\times$ 192, LayerNorm, Linear(192, 512), GELU, Linear(512, 256). Projector: Linear(256, 1024), BatchNorm, GELU, Linear(1024, 128). Loss: $(1-\lambda)\,\mathscr{L}_{\text{inv}} + \lambda\,\mathscr{L}_{\text{SIGReg}}$, $\lambda = 0.05$; SIGReg with 256 random directions and 17 integration knots on $[-5,5]$. Views: 2 global (50–100% contiguous span) and 6 local (15–50%), 15% token masking. Optimization: 1,500 steps (1,000 for the per-family encoders), batch 256, peak learning rate 10^{-3}, 5% linear warm-up, cosine decay, AdamW (weight decay 0.05) for dense parameters and sparse Adam for the embedding bag. Next-token baseline: 1,024 random prefixes per batch, softmax over the whole vocabulary.

Evaluation details. Cross-system precision@10 excludes neighbours from the query's own system and thread and averages over 3,000 random linked records. Probes are multinomial logistic regressions. ρ and R^2 use the top 32 dimensions after whitening, with the between-group covariance corrected for its finite-sample bias. For the alignment test, the 126 states observed by both families are split in half; anchors come from one half, test pairs (1,000) from the other. The per-direction analysis of the retrained encoder fits one rotation and one global scale on 2,000 day-1 records embedded by both models and tests it on 6,000 others, in the centred (not whitened) space; its variance-weighted R^2 is 73% (88% on the top 8 directions). The second retraining test in §6.6 is a separate run: a retrained encoder (different seed, no hidden records), a rotation fitted on 256 anchors in a whitened 64-dimensional space without scale. For the held-out test, threads are drawn at random among linked records; TF-IDF is fitted on the training records only. For the state probes and Figure 6, TF-IDF is refitted on the day-2 corpus.

Masking. Customer account and organization names, the distinctive words in them (words shared by several customer names, such as "payments", are kept), customer web domains, contact names, email addresses and identifiers of records, cases, requests and long numbers are replaced with one placeholder token before tokenization. The second mask also replaces the company's own employees' names (every word of three letters or more in a name), email handles and chat real and display names; it changes 86% of the linked records.

B Details of the additional experiments

Pretrained backbone. all-MiniLM-L6-v2 with mean pooling and normalized outputs, applied to each record's raw text (at most 256 word pieces). Head: LayerNorm(384), Linear(384, 512), GELU, Linear(512, 256); projector and loss as above ($\lambda = 0.05$); 800 steps, batch 256, eight views per anchor (two global and six local, constructed identically here because a frozen embedding cannot be cropped), each a record drawn with replacement from the anchor's state, with 10% feature dropout and Gaussian noise of standard deviation

0.02; AdamW, peak learning rate 10^{-3}, cosine decay. For the held-out test the head is trained on the training threads only.

Months. Month states are (customer, calendar month) pairs; week states use ISO weeks. The month encoder uses the recipe of the main comparison with month states as positives, trained on the training threads of the held-out split. Next-month probes use states from the day-2 corpus embedded by the day-1 encoders, standardized, reduced to 32 principal components and fitted by logistic regression holding out one customer at a time; a transition links a month to the next calendar month of the same customer when both have records. The dynamics models are fitted per held-out customer on the other customers' transitions, in the 32 leading principal components of the standardized states: ridge regression from a state to its change, with $\alpha = 10$ or with $\alpha \in \{0.1, 1, 10, 10^2, 10^3, 10^4\}$ chosen by leave-one-customer-out validation within the training customers, and a single factor k that minimizes $\|s_{t+1} - k\, s_t\|^2$.

Two channels. For the state probes, each channel's state matrix is standardized per dimension and divided by the square root of its dimension before concatenation, so both channels carry the same total variance. For the operational primitives and for a reader's own two channels, the two L2-normalized embeddings are put side by side. In the memory store, the concatenated rows put the L2-normalized good-view and pretrained embeddings side by side before the store's PCA map; the routed store keeps the pretrained rows as content and the good-view rows, with the same time code, as retrieval keys, and embeds questions with both encoders.

Primitives. Events come from Era's ground truth: tickets with urgent or high priority (by creation date) and changes of an opportunity's stage (by the date of the change). Drift uses weekly states with at least three records; a week is scored if the customer has at least one weekly state in the previous eight weeks, against the mean of up to four of them. The volume score is the absolute log ratio between the week's number of records and the mean of the same previous weeks. Misalignment uses customer-months with at least two records from Salesforce or Gong and at least two from Zendesk or Jira; a month is at risk if the customer's worst ticket that month is urgent or high and its latest opportunity is in an open stage (prospecting, qualification, proposal or negotiation). Similar situations use customer-quarter states with at least five records, centred, and cosine similarity; a quarter's situation is its ticket bucket (none, one or two, three to five, six or more) and its deal-stage group (open, closed won, closed lost or no open deal). Chance is the average share of other customers' states that share the query's situation.

Anchored questions. The questions of Appendix F, with the customer's name replaced by "this customer". Each question is anchored by one record of its customer-quarter, drawn at random, with a different draw for each seed, among the records of held-out threads (§6.2) that come from a system other than the one that answers the question: Zendesk for tickets and priority, Gong for calls and Salesforce for the deal stage. Questions without such a record are dropped, which leaves 127 of the 144 test questions and 393 training questions. With names masked, every customer name, employee name and identifier is masked in the stored records (the second mask of §6.3); with names kept, the records are unchanged. The state channel is a good-view encoder trained on the stored records with the held-out threads hidden (one run), and the detail channel is all-MiniLM-L6-v2 applied to the same text. Every stored row is a content vector (the state, detail or concatenated embedding mapped by PCA, a time code and a one-hot code of the system; for the provenance-only reader, the time and system codes alone) and a retrieval key, and every key ranks the records of the question's quarter first. *Neighbours*: the remaining order is the cosine similarity of the finding channel's content alone, with the anchor's own content as the query; the anchor itself is among the records found, and its own thread supplies 19% of them on average with names masked. *Placement*: each linked training

record is keyed by its customer-quarter; each record of a held-out thread, and the anchor, by the customer whose centroid of linked training records in the finding channel is nearest, with the record's quarter for stored records and the question's quarter for the anchor. *Link oracle*: every record, the anchor included, is keyed by its true customer-quarter. Placement and oracle keys get small random noise, fixed for each seed, so that the reader sees 16 random records of the key's customer-quarter, the same for every question about it; customer-quarters with fewer than 16 records are filled with records of other states, which is why the oracle's records are not all from the right customer. The reader's slots are conditioned on the question text, which gives the kind of fact and the quarter but not the customer. Pre-training uses the same construction, with a random record of a training thread as the anchor, "What happened with this customer?" as the question and targets built from the links of training threads only.

Actions and outcomes. Per customer and calendar month of the day-2 corpus: *sales call*, a call in Era's ground truth that started in the month; *leader*, a linked email or chat message written by the CEO or the head of engineering (authors matched to employees by email address or chat id); *CRM task*, a Salesforce task record; *engineering escalation*, a Jira issue record linked to the customer; *support reply*, a Zendesk comment written by an agent (an author who comments on the tickets of at least three customers; four authors qualify); *outbound email*, an email on the customer's threads written by an employee. Each is a 0/1 indicator; the *system mix* of a month is the share of the customer's linked records in it that each of the seven systems wrote. Outcomes: *deal progress*, an opportunity of the customer moving to a later stage or to closed won in the month; *escalation*, an urgent or high-priority ticket created in the month; a customer-month is *in trouble* when it has an escalation and one of the customer's opportunities is in an open stage at the end of the month (§6.10 uses the stage of the customer's latest opportunity). The 396 transitions are those between consecutive months of the 21 customers; the 398 of §6.9 also include prospects. The action-conditioned predictor works in the 32 leading principal components of the month states, which are standardized over all months (an unsupervised step) and whose components are fitted without the held-out customer; it standardizes the indicators, scales them to the spread of the state components, and chooses the ridge strength from $\{1, 10, 10^2, 10^3, 10^4\}$ by leave-one-customer-out validation within the training customers. Outcome models, and the models that predict each action from the state, are logistic regressions ($C = 0.5$) on the 16 leading components of each representation's month state, fitted without the held-out customer, holding out one customer at a time. Intervals are percentiles of 2,000 bootstrap resamples of customers, with the predictions held fixed.

Event windows and decision simulation. Event windows contain the customer's linked records in the 14 days before and after the action, without any record of the action's kind (all call transcripts and calendar events, all Jira issues, all records written by a leader), and need at least two records on each side; a leader event is a leader's message after 14 days without one. Each event is paired, when one is found in 20 random draws, with a time drawn from the same customer's records with no action of that kind within 14 days (for leaders, no leader message) and at least two records on each side; 29 of the 66 leader events found one, and every call and escalation did. The state of a window is the mean of its records' good-view embeddings and the shift is $1 -$ cosine between the two windows; the adjusted difference is the coefficient of the event indicator in a least-squares fit of the shift on the indicator and the logarithms of the two windows' record counts. Intervals are percentiles of 2,000 bootstrap resamples of customers. The predictor of the window after from the window before is the ridge regression above, in 32 principal components, with and without a one-hot indicator of the action, holding out one customer at a time. Decision simulation standardizes each representation's month states and reduces them to 16 principal components, fitted once, without labels, on the 396 months that have a following month, and compares them by cosine: for each month in trouble, the 60 most similar months of the other customers (any month with a following month) are split by whether the

company intervened that month, and the first 15 of each group give the outcome rates. The outcome model is a logistic regression ($C = 0.5$) on the good-view components and an intervention indicator, fitted on the other customers' months and evaluated with the indicator set to 1 and to 0. Intervals are percentiles of 300 bootstrap resamples of customers, repeating the neighbour search and refitting the outcome model in each. Balance is the share of each group's months that were themselves in trouble, or had an urgent ticket; the propensity AUROC is that of a logistic regression predicting intervention from the 16 components, holding out one customer at a time.

Access control with a backbone. The canary records are embedded by the backbone with their planted text. Each reader's head is trained on that reader's readable training records only (800 steps). The public base is a head trained (800 steps) on the 534 customer-linked records that all twelve employees may read, outside the held-out threads, and then fine-tuned for 800 steps on the reader's records. The scope test samples, with a fixed seed, as many records from all systems as the reader may read, trains the scratch encoder on them with the held-out threads hidden and a vocabulary built from the sample, and places the reader's held-out readable records against the reader's readable training states.

C Access-control policy

Readers are derived from each record's system, channel and sender with the role-based policy below (`cwm/acl.py`); the second table gives the share of customer-linked records each evaluated reader may read, overall and by system. For the leak tests, a unique codename and the sentence "Privately we expect them to churn to a competitor next quarter." were appended to 150 training records of five customers in Salesforce and Gong, which neither the support engineer nor the software engineer may read. Each reader's encoder is trained with the same recipe as the main comparison (good views, one seed, a quarter of the threads held out), on the records that reader may read and with a vocabulary built from them.

Table 14: The access policy for the simulated company, and what each evaluated reader may read.

Records	Who may read them
Slack, public channels (including customer channels)	everyone
Slack, private channels (leadership, exec-staff)	leadership
Slack, direct and group messages	their members
Jira (issues)	everyone
Zendesk (support tickets)	support, success, engineering, product, leadership
Salesforce (CRM)	sales, success, finance, leadership
Gong (call transcripts)	sales, success, product, leadership
Gmail	the sender, plus sales, success, support, leadership
Google Calendar	the organizer, plus leadership

Reader	all	slack	gmail	zendesk	salesforce	gong	jira	calendar
CEO	100%	100%	100%	100%	100%	100%	100%	100%
Account executive	81%	100%	100%	0%	100%	100%	100%	59%
Support engineer	86%	100%	100%	100%	0%	0%	100%	25%
Software engineer	22%	100%	1%	100%	0%	0%	100%	1%
Financial analyst	9%	100%	2%	0%	100%	0%	100%	0%

D All results of the main comparison

Table 15: Per-seed results of the main comparison on the training records: cross-system precision@10, customer probes and ρ.

Objective	Seed	Same customer	Same state	M→R	R→M	ρ_{state}
Good views	0	0.953	0.389	0.883	0.978	0.72
Good views	1	0.952	0.384	0.878	0.937	0.74
Good views	2	0.947	0.397	0.881	0.866	0.73
Bad views	0	0.140	0.050	0.244	0.142	0.17
Bad views	1	0.137	0.049	0.221	0.141	0.16
Bad views	2	0.132	0.046	0.218	0.119	0.17
Next-token	0	0.123	0.041	0.261	0.184	0.18
Next-token	1	0.129	0.042	0.243	0.213	0.18
Next-token	2	0.143	0.046	0.240	0.232	0.18

E State tracking and forecasting

For each customer-quarter, the state vector is the mean of the L2-normalized embeddings of its records, from the frozen day-1 encoders applied to all records through day 2. Probes are standardized, reduced to 32 principal components and fitted by logistic regression, holding out one customer at a time, so that no probe can memorize a customer. "Now" predicts a quarter's facts from its own state; "next" predicts the following quarter's facts from the current state (165 transitions). The majority class is the share of the most common answer. Oracle persistence copies the current quarter's true facts and therefore reads the ground truth directly.

Table 16: Accuracy of linear probes on state vectors, holding out one customer at a time.

	This quarter					Next quarter				
Representation	tickets	priority	call	stage	mean	tickets	priority	call	stage	mean
Good views	0.59	0.55	0.72	0.43	0.57	0.32	0.35	0.68	0.35	0.43
Bad views	0.72	0.71	0.97	0.41	0.70	0.48	0.49	0.62	0.35	0.48
Next-token	0.71	0.68	0.94	0.34	0.67	0.55	0.52	0.64	0.27	0.49
TF-IDF + LSA	0.70	0.67	0.82	0.41	0.65	0.52	0.53	0.60	0.39	0.51
Random encoder	0.64	0.63	0.85	0.36	0.62	0.49	0.46	0.68	0.29	0.48
Majority class	0.57	0.57	0.71	0.29	0.54	0.51	0.51	0.65	0.30	0.49
Oracle persistence			—			0.71	0.59	0.69	0.70	0.67

F Reading the store with memory tokens

Each linked record is stored once as a row: its embedding mapped to 112 dimensions by PCA whitening, a 16-dimensional sinusoidal time code and a one-hot code of its source system. A Memorilla-style module [9] turns a question, embedded by the store's encoder, into 16 query-conditioned slots that attend, through two Perceiver-style cross-attention blocks, either over the whole store or over 16 retrieved rows: the rows of the question's quarter whose content in the retrieval channel is most similar to the question's. (An earlier version of this paper ranked whole rows, in which the time code outweighed the content and only 24%–35% of the

rows retrieved belonged to the asked customer; ranking content within the quarter raises this to 65%–71% and leaves the conclusions of §6.8 unchanged. The rows of Table 17 for the reader over the whole store come from that earlier version; they do not retrieve.) The slots are averaged and matched against fixed embeddings of the 18 possible answers from a pretrained sentence encoder (all-MiniLM-L6-v2), so the reader is a classifier with a frozen output layer rather than a language model. Questions are generated from Era's ground truth for each customer-quarter: the number of support tickets, the worst ticket priority, whether a sales call happened and the deal stage. Customer-quarters are split 70/10/20 into training, validation and test (36 test customer-quarters, 144 test questions). The module is first pre-trained for 250 steps to recover what the store holds about a customer and date, including the share of records from each system and their volume, then trained on the questions for 12 epochs, keeping the checkpoint that does best on validation. With the pretrained store, questions are also embedded by the pretrained encoder, which changes the closed-book baseline. Runs are deterministic for a given seed and thread count but not across thread counts.

Table 17: Accuracy of the memory reader on the 144 test questions (mean over seeds); the answer layer is frozen.

				Next quarter: store	
Store	Reader	Seeds	Accuracy	not updated	updated
Good views (state)	closed book (no store)	4	0.618	0.547	
	16 tokens over the 16 retrieved rows	4	0.658	0.594	0.522
	16 tokens over the whole store	1	0.639	0.576	0.576
	the same, not pre-trained	1	0.639	0.523	0.523
Pretrained (detail)	closed book (no store)	4	0.642	0.573	
	16 tokens over the 16 retrieved rows	4	0.712	0.577	0.554
Two channels, concatenated	closed book (no store)	4	0.635	0.565	
	16 tokens over the 16 retrieved rows	4	0.722	0.612	0.548
Two channels, routed	16 tokens over the 16 retrieved rows	4	0.701	0.593	0.481